SIMATS ENGINEERING

Saveetha Institute of Medical and Technical Sciences

CSA1737 – ARTIFICIAL INTELLIGENCE

ASSIGNMENT 2 (CO2 – Unit II: Searching Techniques)

Informed Search, Online Search, Adversarial Search, Constraint Propagation and Optimisation: Predict, Measure, Diagnose and Redesign

Item	Details
Student name	**Saai Vardhan D**
Register number	**192525205**
Course	CSA1737 – Artificial Intelligence (L-T-P-C: 3-0-2-4)
Assignment	Assignment 2 – CO2 (Individual, 50 marks, Tasks 1–7)
Language / tools	Python 3, Matplotlib; run in a Linux container
Date of submission	22 October 2026

Method followed in every task: Predict (written to a predictions file BEFORE the run) → Design → Implement → Measure (operation counts first, time second) → Compare prediction with observation → Evaluate. Where a prediction was wrong, the report says so and explains why. All numbers come from the code in the appendices; random seeds are reported.

Contents and Summary of Findings

Task	Title	Marks
1	Uniform-cost, Greedy best-first and A* under different map conditions	6
2	When is an online search agent an appropriate choice?	4
3	Engineering the behaviour of alpha-beta pruning	6
4	Minimax and the cost of examining every possibility	6
5	Deriving a constraint-propagation algorithm (exam timetabling)	7
6	Exhaustive search, local search and combinatorial growth (TSP)	8
7	Day-4: diagnose, redesign and validate (minimax)	13
	Total	**50**

Key findings (all measured)

Task 1. There is no universally best algorithm. On open road A* (tie-break on h) expanded 178 cells against 3,533 for UCS (20× fewer) and stayed optimal; Greedy was cheapest (123) but optimal in only 2/30 maps and 49 % too costly on the rough-terrain map. A* with FIFO tie-breaking lost the whole advantage (3,104). **Recommendation: A∗ with h-tie-breaking.**

Task 2. Online DFS declares an unreachable target after exactly 2(cells−1) moves (392 on 21×21); LRTA* never terminates. With 20 facilities and static maps, explore once, store the map and plan: it saves 16 % (M = 2) to 31–44 % (M = 50) of the moves against repeating online DFS.

Task 3. Same b and d, very different work: for b = 4, d = 8, alpha-beta evaluated 511 leaves (best order), 7,141 (random) or all 65,536 (adversarial order, 0 cutoffs). Branching factor and depth alone do not determine the cost.

Task 4. Plain minimax nodes match $(b^{d+1}-1)/(b-1)$ exactly while no column fills or game ends; the practical limit at 1 s per move is depth 5 on 7×6 (Connect-Four), depth 6 on 6×5 and 7 on 5×5. 82 % of the time is spent in the evaluation function.

Task 5. My derived rule (revise + re-queue) solves one instance by propagation alone and detects failure on another with zero search. MAC cut mean nodes from 100,000 (capped) to 560 at n = 60; the pigeonhole and hidden-core instances show where propagation is not enough.

Task 6. Exhaustive tours = (n−1)! exactly (verified n = 5–12); time grows 10–12× per extra customer. At a 2 s response limit, exhaustive search is deployable up to n = 10 (0.28 s); n = 11 takes 2.8 s. 2-opt with 10 restarts reached the optimum in 197 of 200 runs (98.5 %) at n ≤ 12.

Task 7. Redesigned minimax (alpha-beta + centre-first ordering + transposition table + iterative deepening) gave identical root values in 3,360 tests with **70× fewer evaluation calls at depth 6**, reached depth 10 instead of 5 within 1 s on 7×6, and won 13 of 20 games (2 draws) against the plain search under equal time.

Task 1 – Greedy Best-First and A* Under Different Map Conditions (6 marks)

1.1 Problem set-up

The campus is a 61 × 61 grid. The shuttle moves in the four compass directions. A road cell costs 1, a rough cell (gravel, speed bump zone) costs 5, and # is blocked (construction or closed road). Three algorithms are compared: **UCS** (expands by lowest g), **Greedy best-first** (lowest h) and **A*** (lowest f = g + h). Every algorithm uses the same graph interface and the same counters: nodes expanded, nodes generated, maximum frontier size, path cost and a flag for optimality (compared with UCS, which is optimal because all costs are positive).

1.2 My admissible heuristic and the relaxed problem

> **Heuristic: $h(n) = |x_n - x_{goal}| + |y_n - y_{goal}| \times c_{min}$, $c_{min} = 1$ (Manhattan distance)**
>
> **Relaxed problem.** Remove every obstacle and let every cell cost the minimum road cost 1. The exact solution of the relaxed problem (shortest path on an empty grid with unit steps) is the Manhattan distance. Removing walls can only add routes and lowering all costs to 1 can only lower the cost, so the relaxed optimum is never larger than the real optimum: $h(n) \le h^*(n)$. Hence **h is admissible**.
>
> **Consistency.** One move changes x or y by exactly 1, so h changes by at most 1, while the real step cost is at least 1: $h(n) \le c(n,n') + h(n')$. So A* never needs to re-open a node, and the first time it removes the goal the path is optimal.
>
> **Check by program.** On four maps I computed the true cost-to-goal h* of every cell by reverse Dijkstra and counted violations of $h \le h^*$: 3,666 + 3,721 + 3,504 + 1,799 cells checked, **0 violations**.

1.3 Predictions (written before any run)

The complete text is saved as `t1_predictions.txt` and reproduced in Appendix B. The reasoning behind each is algorithmic:

- **Open map (5 % obstacles).** UCS explores a diamond of all cells with $g \le C^*$, so nearly all free cells (3,000–3,500). Greedy follows h straight to the goal (about the path length, 120–200). A* with ties broken towards smaller h expands a narrow band (120–400); with FIFO ties every cell inside the rectangle has $f = C^*$, so it degrades towards UCS (1,500–3,300).
- **Maze.** The straight-line heuristic points through walls and gives little guidance: UCS 50–100 % of the cells, A* 60–100 % of UCS, Greedy 40–90 % of the cells; all return the same cost because the path is unique.
- **Concave trap.** Greedy rushes into the cup, fills it, then escapes: 300–900 expansions and a path that is not optimal; A* 40–70 % of UCS.
- **Rough terrain.** Greedy ignores cost and walks through the rough band (cost ≥ 40 % above optimal). Because h assumes cost 1 per cell, it under-estimates rough cells badly, so A* should be close to UCS (70–100 %).

1.4 The three designed maps

Map 1 (open road) and **Map 2 (perfect maze)** are the two deliberately contrasting maps; **Map 3 (concave trap)** is my edge case; I added **Map 4 (rough terrain)** because the 95 %-open-road statement makes cost-

awareness important. Maps 1 and 2 are averaged over 30 random seeds (0–29); Maps 3 and 4 are single hand-designed layouts.

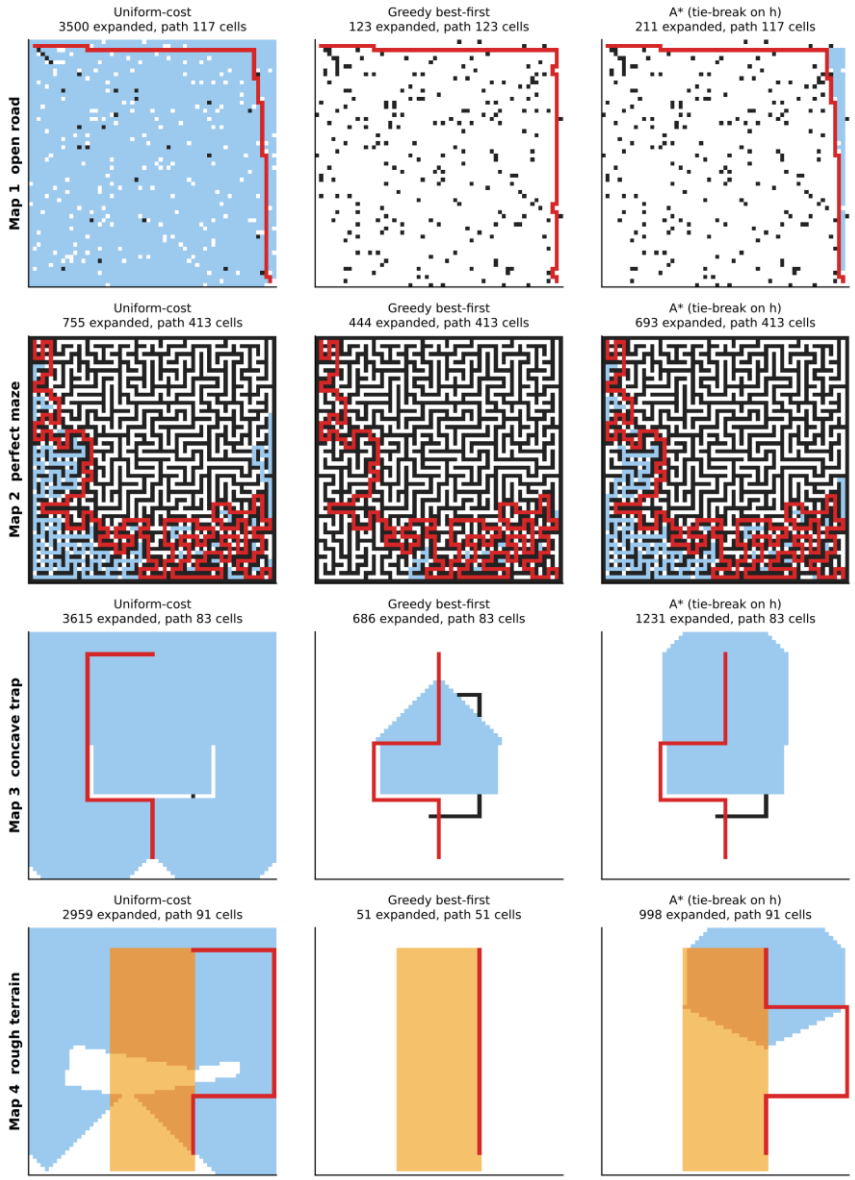


Figure 1.1 Expanded cells (light blue; orange = rough cells expanded) and the path found (red) for the three algorithms on each map. Maps 1–2 show seed 0.

1.5 Results

Map	Algorithm	Expanded	Generated	Max frontier	Cost	Optimal	ms
1 Open (mean of 30)	UCS	3,533	3,535	62	116.0	30/30	6.8
	Greedy	123	332	210	120.3	2/30	0.6
	A* (tie-break h)	178	433	256	116.0	30/30	0.5
	A* (FIFO ties)	3,104	3,318	215	116.0	30/30	6.4
2 Maze (mean of 30)	UCS	1,046	1,048	6	632.7	30/30	1.7
	Greedy	778	790	13	632.7	30/30	1.3
	A* (tie-break h)	988	991	6	632.7	30/30	1.7

Map	Algorithm	Expanded	Generated	Max frontier	Cost	Optimal	ms
	A* (FIFO ties)	992	995	6	632.7	30/30	1.6
3 Concave trap	UCS	3,615	3,632	72	82	yes	6.6
	Greedy	686	789	104	82	yes	1.5
	A* (tie-break h)	1,231	1,343	113	82	yes	2.5
	A* (FIFO ties)	1,763	1,907	145	82	yes	3.2
4 Rough terrain	UCS	2,959	3,093	135	90	yes	5.5
	Greedy	51	152	102	134	**NO (+49 %)**	0.1
	A* (tie-break h)	998	1,162	165	90	yes	2.1
	A* (FIFO ties)	1,316	1,454	139	90	yes	2.6

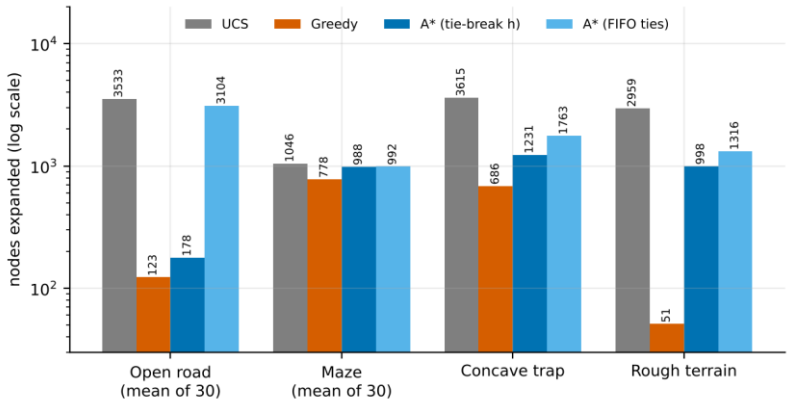


Figure 1.2 Nodes expanded per algorithm and map (log scale). Open-road and maze bars are means of 30 maps.

1.6 Prediction versus observation

Item	Predicted	Observed	Verdict
Open: UCS	3,000–3,500	3,533	Marginally above range (the diamond also contains the cells beside the path)
Open: Greedy	120–200; cost 0–10 % above	123; +3.7 %	Correct
Open: A* / A* FIFO	120–400 / 1,500–3,300	178 / 3,104	Correct
Maze: UCS, A*, Greedy	50–100 % of cells; 60–100 % of UCS; 40–90 %	58 %; 94 % of UCS; 43 %	Correct
Trap: Greedy expansions	300–900	686	Correct
Trap: Greedy cost	not optimal	**optimal (82)**	**WRONG**
Trap: A* vs UCS	40–70 %	34 %	**Wrong** (A* was better than predicted)
Rough: Greedy cost	≥ 40 % above optimal	+49 %	Correct
Rough: A* vs UCS	70–100 %	34 %	**Wrong** (A* was better than predicted)

Why the misses happened. (i) On the trap map the cup is escaped along the same side that the optimal route uses, so the long wander inside the cup (686 expansions for an 83-cell path, 8×) costs effort but not path length. A narrower cup whose exit lies on the wrong side would be needed to make Greedy's path

longer; I did not test this. (ii) The rough band in Map 4 only has to be avoided, not crossed, and h = Manhattan still points away from the band's high-g region, so A* prunes everything whose g + h exceeds 90. I had assumed the weak heuristic would cripple it; it is weak inside the band but sharp outside.

What the maps show about each algorithm. UCS expansions depend only on the cost of the solution (a disc of radius C*), so it is blind to direction. Greedy expansions depend only on how well h guides it: tiny on open ground (123 against 3,533), large where h misleads (maze, trap), and its cost is unconstrained (rough terrain: 134 against 90). A* sits between: it never loses optimality (every row is optimal) and its expansions shrink as h becomes more informative (open map: 20× fewer than UCS). **The tie-breaking rule matters as much as the heuristic**: on Map 1 the same A* with FIFO ties expanded 3,104 cells, 17.4× more than with ties broken towards smaller h, because with a consistent unit-cost heuristic all cells inside the start–goal rectangle share f = C*.

1.7 Design decision: 95 % open-road trips

Algorithm	Expected expansions per trip	Expected excess cost
UCS	3,483	0.00 %
Greedy	142	4.36 %
A* (tie-break h)	223	0.00 %
A* (FIFO ties)	3,017	0.00 %

Table 1.2 Workload model: 0.95 × (mean over 30 open maps) + 0.05 × (mean of maze, trap and rough maps).

Recommendation: A∗ with Manhattan heuristic and ties broken towards smaller h. It is optimal on every map (UCS-quality routes) and, on the 95 % open-road trips, expands about 16× fewer cells than UCS (223 against 3,483 expected). Greedy would expand slightly fewer (142) but returns a route 4.4 % longer on average, and the 5 % of trips that go around obstacles are exactly where Greedy's cost is unbounded (+49 % on the rough map). A shuttle company pays for travel time on every trip, while the extra 80 expansions cost microseconds, so guaranteed optimality is the better trade. Limitations: A* keeps a larger frontier than UCS on Map 1 (256 against 62), and its advantage collapses if ties are broken carelessly (FIFO) or the heuristic is weak (Map 4 inside the rough band). If route cost were very uncertain, I would raise c_min only when the minimum cell cost really is larger than 1 (a stronger admissible heuristic).

Task 2 – When Is an Online Search Agent an Appropriate Engineering Choice? (4 marks)

2.1 Agents and experimental design

The rover knows its own coordinates and the dock coordinates, senses only which of the four neighbouring cells are free, and learns the map as it moves. Two agents were built: **online DFS** (move to an unvisited free neighbour, otherwise backtrack along its own trail; it terminates with failure when it is back at the start with nothing unvisited) and **LRTA∗** (keeps a learned estimate H(s), initial H = Manhattan distance; at each step moves to the neighbour minimising c + H and raises H(s) to that value; H persists across trials). Competitive ratio **CR = moves ÷ optimal path length with the full map**. Revisited cells = moves into a cell already visited.

Mazes: perfect mazes on grids of side 11, 21, 31 and 41 (49, 199, 449 and 799 free cells) generated by randomised depth-first carving, 30 mazes (seeds 0–29) per setting. Dead-end density is varied by 'braiding' (opening a fraction of the dead ends into loops). Target distance: the free cell at the 20th percentile, median and maximum of the true distance from the start. Because DFS-carved mazes have very few branches, I added a second family of 'bushy' mazes (randomised Prim) after seeing the first results.

2.2 Predictions (written before any run)

No.	Prediction
P1	Perfect 21×21 maze: online DFS far-target CR 3–6; near target CR 5–10 with large variance.
P2	LRTA* first trial CR 3–8; over repeated trials CR → 1.0 within about 10–20 trials; online DFS never improves.
P3	Unreachable target: online DFS stops after exactly 2(cells−1) moves (440 on 21×21); LRTA* never stops.
P4	CR of online DFS roughly doubles from 11×11 to 41×41.
P5	Removing dead ends (braiding) increases the CR of online DFS.
P6	With static maps, explore-once-then-plan breaks even against repeated online DFS at M = 2 missions and against LRTA* at M of about 3–10.

2.3 Results

Experiment 1 – target distance (21×21 perfect mazes, 30 seeds)

Target	Optimal length	DFS moves	DFS revisits	DFS CR	LRTA* trial 1 moves	LRTA* trial 1 CR	LRTA* trial 10 CR
Near (20th pct)	35.3	108.8	36.7	3.12	107.8	3.10	1.18
Middle	76.0	114.9	19.5	1.53	562.5	7.45	1.47
Far	136.5	198.4	30.9	1.46	2,185.7	15.81	2.11

Experiment 2 – maze size (far target)

Size	Free cells	Optimal	DFS moves	DFS CR	LRTA* t1 moves	LRTA* t1 CR	LRTA* t10 CR	Prim: DFS CR
11×11	49	38.0	47.5	1.26	179.7	4.77	1.24	2.86
21×21	199	136.5	198.4	1.46	2,185.7	15.81	2.11	4.86
31×31	449	274.3	431.4	1.59	6,562.9	24.03	2.49	7.32
41×41	799	463.9	845.5	1.84	15,921.9	34.70	2.94	9.54

Experiment 3 – dead-end density (31×31, far target)

Braid fraction	Dead ends	Optimal	DFS moves	DFS CR	LRTA* t1 CR
0.00	24.3	274.3	431.4	1.59	24.03
0.25	18.1	168.5	386.5	2.48	25.66
0.50	12.0	128.9	339.6	2.92	24.39
0.75	5.9	104.9	306.6	3.14	17.59
1.00	0.0	88.7	244.3	2.88	15.35

Experiment 4 – unreachable target (target sealed by walls)

Size	Reachable cells	DFS moves	2(cells−1)	DFS verdict	LRTA* (budget 60 × cells per trial)
11×11	47	92	92	30/30 correct	budget exhausted 30/30
21×21	197	392	392	30/30 correct	budget exhausted 30/30
31×31	447	892	892	30/30 correct	budget exhausted 30/30

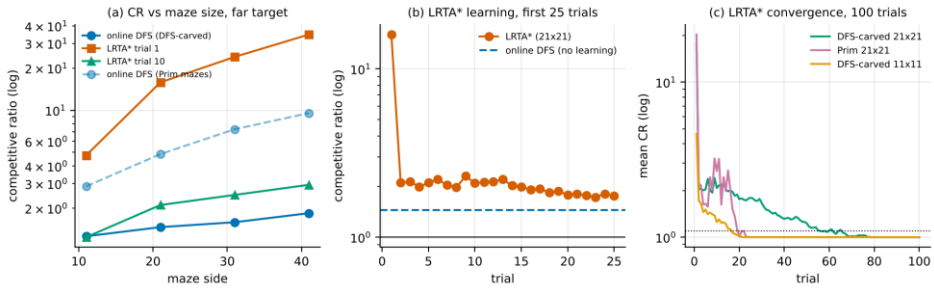


Figure 2.1 (a) competitive ratio against maze size; (b) LRTA learning on 21×21 mazes (mean of 30); (c) LRTA* convergence over 100 trials (mean of 20 mazes).*

2.4 Prediction versus observation

Pred.	Observed	Verdict
P1 DFS CR 3–6 (far), 5–10 (near)	Far 1.46, middle 1.53, near 3.12 on DFS-carved mazes. On bushy (Prim) 21×21 mazes: far 4.86, near 12.65.	**Wrong on DFS-carved mazes, right on bushy mazes.** A DFS-carved maze is mostly one long winding corridor with few branches, so there is little to waste; the optimal path is already 69 % of all cells.
P2 LRTA* trial 1 CR 3–8; CR → 1 in 10–20 trials	Trial 1 CR 15.8 (21×21), 4.8–34.7 over sizes. Trial 2 already 2.1; mean CR ≤ 1.10 only after 56 trials (21×21 DFS-carved), 20 trials (Prim 21×21), 17 trials (11×11).	**Wrong in magnitude.** The first trial is very costly (the heuristic points through walls), then learning is fast to about 2× and slow to 1.0. DFS: no learning, as predicted.
P3 DFS 2(cells−1) moves; LRTA* never stops	92 / 392 / 892 exactly; LRTA* exhausted its budget in 30/30 mazes.	**Correct, exact.**
P4 CR doubles 11→41	1.26 → 1.84 (×1.46); Prim mazes 2.86 → 9.54 (×3.3)	Direction right; magnitude depends on maze structure.
P5 braiding raises DFS CR	1.59 → 3.14 at 0.75 braid, 2.88 at 1.0	**Correct** (small fall at 1.0).
P6 break-even M = 2 vs DFS	Explore once + plan: 330 against 392 moves at M = 2 (21×21)	**Correct.**

2.5 When is an online agent a reasonable engineering choice?

From the evidence the online agent is reasonable when **all** of the following hold:

- **The map really is unknown and each mission happens once (or the map changes between missions).** With one mission, online DFS on a 21×21 maze costs 196 moves for a 134-move optimum (CR 1.46), which is acceptable.

- **The environment is small or corridor-like.** CR of online DFS stays below 2 on DFS-like corridors up to 41×41 (1.84), but on bushy mazes it climbs to 4.9 (21×21) and 9.5 (41×41) for far targets and to 12–23 for near targets, where the blind search explores nearly everything before it stumbles on a close dock.
- **Failure must be detected cheaply.** Online DFS is complete and proves unreachability in exactly 2(cells−1) moves; LRTA* cannot, so LRTA* is only suitable when the target is guaranteed reachable.

I would stop recommending online search when (a) the same facility is visited more than once, (b) the maze is large and bushy so that CR is above about 5, or (c) the cost of a wasted move (battery, time) is high and the unreachable-target case occurs often, since the whole reachable map must be walked twice. LRTA* is attractive only when many repeated trials in an unchanging environment are expected AND the first-trial cost (CR 16–35, 2,186 moves on 21×21) is tolerable; its 20–56 trials to converge to CR ≈ 1 are slow.

2.6 Changed condition: 20 rovers, 20 facilities, maps fixed for months

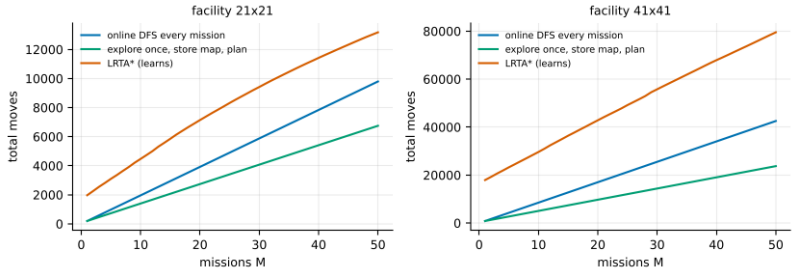


Figure 2.2 Total moves for one facility over M missions: repeat online DFS, LRTA (learning), or explore once and plan on the stored map.*

Facility	M	DFS every mission	LRTA* (learns)	DFS once + stored map	Saving vs DFS
21×21	2	392	2,257	330	15.9 %
21×21	10	1,959	4,468	1,399	28.6 %
21×21	50	9,795	13,178	6,747	31.1 %
41×41	2	1,702	19,229	1,318	22.6 %
41×41	10	8,509	29,627	5,053	40.6 %
41×41	50	42,545	79,517	23,729	44.2 %

Revised recommendation. The unchanged maps remove the reason for online search after the first mission. Each rover should run online DFS **once**, store the map it discovered, and plan every later mission on the stored map (A* with the Manhattan heuristic from Task 1), re-using online DFS only when a sensed cell contradicts the map. This costs one exploration and then the optimal path each time (saving 16 % already at M = 2 and 31–44 % over 50 missions). LRTA* is worse than simply repeating DFS for the first 50 missions on both facility sizes (13,178 against 9,795 on 21×21) because of its expensive first trial, so I no longer recommend it here. Across 20 facilities the saving multiplies, and the stored map can also be checked once to identify unreachable docks (2(cells−1) moves) instead of paying that price on every mission.

Task 3 – Engineering the Behaviour of Alpha-Beta Pruning (6 marks)

3.1 Algorithms and counters

```
function ALPHABETA(node, depth, alpha, beta, isMax)        // counts: nodes += 1 on every call
    nodes += 1
    if depth == 0 or node is a leaf:  leaves += 1;  return value(node)
    if isMax:
        v = -inf
        for child in children(node):                       // the ORDER of this loop is the whole story
            v = max(v, ALPHABETA(child, depth-1, alpha, beta, False))
            alpha = max(alpha, v)
            if alpha >= beta:  cutoffs += 1;  break        // remaining children are never visited
        return v
    else:                                                  // MIN node: symmetric, uses beta = min(beta,
v)
        v = +inf
        for child in children(node):
            v = min(v, ALPHABETA(child, depth-1, alpha, beta, True))
            beta = min(beta, v)
            if alpha >= beta:  cutoffs += 1;  break
        return v
```

Minimax (same recursion without alpha, beta and the break) is used as the baseline. Every alpha-beta run is checked to return the same root value as minimax. A **node** is every call of the function (root included); a **cutoff** is one execution of the `break`.

3.2 The three designed trees

All trees are uniform (every internal node has b children, all leaves at depth d), the root is a MAX node. Leaf values are integers I assign so that the ordering of the children is exactly what each experiment needs:

- **Tree 1 – many cutoffs ('best').** At every MAX node the best child is first, and at every MIN node the child that is worst for MAX is first. The search then examines only the 'minimal tree'.
- **Tree 2 – almost no cutoffs ('worst').** I first tried 'worst move first at every node' with distinct leaf values. It still produced cutoffs (11,616 for b = 4, d = 8) because bounds inherited from higher levels cut deeper nodes. The corrected design is a **lexicographic adversarial tree**: leaf values increase from left to right in such a way that at every MAX node the children are in increasing order and at every MIN node in decreasing order, so every new child improves the node's value without ever reaching the other player's bound. Its result is 0 cutoffs.
- **Tree 3 – my own conditions.** (a) *Random order*: leaf values drawn at random (100 trees per setting; the main table shows one tree, the range over all 100 is quoted). (b) *Equal leaf values*: every leaf has the same value, with cutoffs triggered when alpha ≥ beta.

3.3 Predictions (computed before any tree was searched)

For the best case I used the node-type recurrences of the minimal tree. A **PV node** searches all b children, of which the first is a PV node and the other b−1 are CUT nodes; a **CUT node** searches one child (an ALL node) and then cuts the rest; an **ALL node** searches all b children, each a CUT node. With h = remaining depth:

$$T_1(h) = 1 + T_1(h-1) + (b-1)\cdot T_2(h-1),\quad T_2(h) = 1 + T_3(h-1),\quad T_3(h) = 1 + b\cdot T_2(h-1),\quad T^*(0) = 1$$

which gives the Knuth–Moore leaf count **b^{⌈d/2⌉} + b^{⌊d/2⌋} − 1**. For the worst case every child improves the bound, so nothing is refuted and the work equals minimax: b^d leaves and $(b^{d+1}-1)/(b-1)$ nodes. For random order I used the Knuth–Moore/Pearl average of about $b^{3d/4}$ leaves with an allowance of a factor

of 0.5 to 2. For equal values every CUT node is refuted by its first child, so I predicted the same count as the best case.

3.4 Results

b	d	Tree	Nodes	Leaves	Cutoffs	Predicted leaves	Leaves ÷ b^d	Verdict
4	4	best	62	31	21	31	0.121	exact
4	4	worst-first (distinct)	338	253	19	256	0.988	**differs**
4	4	lexicographic	341	256	0	256	1.000	exact
4	4	random	171	113	31	32–128	0.442	in range
4	4	equal	62	31	21	31	0.121	exact
4	6	best	268	127	99	127	0.031	exact
4	6	worst-first (distinct)	4,979	3,671	696	4,096	0.896	**differs**
4	6	lexicographic	5,461	4,096	0	4,096	1.000	exact
4	6	random	1,453	937	320	256–1,024	0.229	in range
4	6	equal	268	127	99	127	0.031	exact
4	8	best	1,098	511	417	511	0.008	exact
4	8	worst-first (distinct)	62,418	45,412	11,616	65,536	0.693	**differs**
4	8	lexicographic	87,381	65,536	0	65,536	1.000	exact
4	8	random	11,256	7,141	2,741	2,048–8,192	0.109	in range
4	8	equal	1,098	511	417	511	0.008	exact
3	8	best / lexi	393 / 9,841	161 / 6,561	152 / 0	161 / 6,561	0.025 / 1	exact
3	8	random / equal	2,366 / 393	1,321 / 161	613 / 152	364–1,458 / 161	0.201 / 0.025	in range / exact
5	6	best / lexi	491 / 19,531	249 / 15,625	180 / 0	249 / 15,625	0.016 / 1	exact
5	6	random / equal	3,702 / 491	2,569 / 249	768 / 180	699–2,795 / 249	0.164 / 0.016	in range / exact

Table 3.1 Every alpha-beta run returned the same root value as minimax. Minimax always visits all b^d leaves with 0 cutoffs (for b = 4, d = 8: 87,381 nodes, 65,536 leaves).

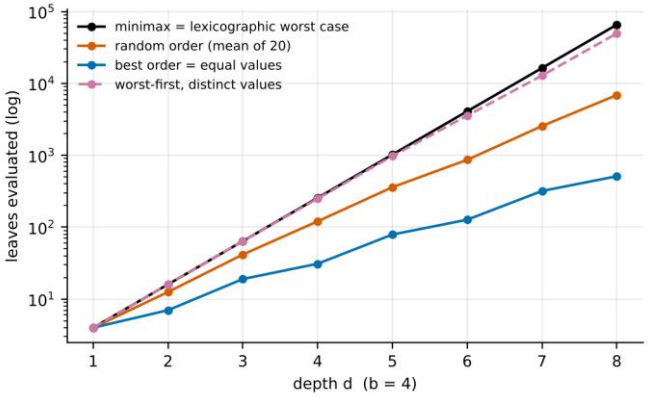


Figure 3.1 Leaves evaluated against depth for b = 4 (log scale). Slope = effective exponent: minimax 1.00, random 0.80, best order 0.56.

d	Minimax	Best	Random (mean of 20)	Worst-first (distinct)	Lexicographic	Equal
2	16	7	13	16	16	7
4	256	31	121	250	256	31
6	4,096	127	865	3,565	4,096	127
8	65,536	511	6,884	49,587	65,536	511

Table 3.2 Depth sweep for b = 4 (leaves evaluated). Effective exponent log_b(leaves)/d at d = 8: best 0.56, random 0.80, minimax 1.00.

3.5 Prediction versus observation and explanation

Matches. The minimal-tree formula was exact in all five best-case settings (31, 127, 511, 161, 249 leaves; 21, 99, 417, 152, 180 cutoffs), the lexicographic tree gave exactly the minimax work with 0 cutoffs, equal values reproduced the best case exactly, and random order fell inside the predicted range in all five settings (leaves 0.11–0.44 of b^d). **Miss.** My simple 'worst child first' tree was predicted to have zero cutoffs and had 19 to 11,616 of them. The prediction ignored that alpha and beta are inherited: a deep MIN node can be refuted by a bound established far above it, even if its own children improve one after another. Producing a true worst case needs the ordering to be adversarial against the inherited bounds too, which is what the lexicographic construction does.

Why one tree needs much more work than another. A cutoff happens when a node's value is already known to be irrelevant to an ancestor: at a MAX node $v \geq \beta$, at a MIN node $v \leq \alpha$. This can only be discovered after a child that is good enough has been searched. If the best child comes first, the first child alone establishes the strong bound, and the remaining b−1 siblings are skipped without being looked at, at every second level; the work becomes about the square root of minimax (511 against 65,536 leaves). If the best child comes last, no sibling ever beats the bound until it is too late and pruning has no effect.

What this says about judging an algorithm by b and d. For b = 4, d = 8 the node count ranges from 1,098 to 87,381, an **80× spread with identical b and d**. The notation O(b^d) is only the worst case, $\Omega(b^{d/2})$ the best case, and the typical case depends on move ordering (random order gives an effective branching factor $b^{0.8}$, about 3.0 instead of 4). Hence evaluating the algorithm from the branching factor and depth alone is not enough; the *ordering heuristic* is part of the algorithm. This motivates the centre-first ordering and iterative deepening in Task 7.

Task 4 – Minimax and the Cost of Examining Every Possibility (6 marks)

4.1 Game, algorithm and evaluation function

The game is **Connect-K with gravity** (K = 4) on W × H boards: a move drops a piece into one of the non-full columns, so b = W on an empty board. Three boards are used: **5×5, 6×5 and 7×6 (Connect-Four)**. The maximiser is the agent ('me'), the opponent minimises. Plain depth-limited minimax is used with no pruning and no move ordering.

How the cut-off is handled. (1) If the last move completed four in a row, the node is terminal with value ±(1,000,000 + remaining depth), so earlier wins score higher. (2) If no legal move exists it is a draw (0). (3) If depth = 0 the **evaluation function** is called once.

> **Evaluation function (my own design)**
>
> Every window of 4 consecutive cells (horizontal, vertical, two diagonals; 69 windows on a 5×5 or 7×6 board) is scored. A window with only my pieces scores S[count] with S = (0, 1, 10, 100, 100000); a window with only the opponent's pieces scores −S[count]; a mixed or empty window scores 0 (nobody can complete it). Plus 3 points for each of my pieces in the centre column and −3 for each of the opponent's, because centre pieces take part in the most windows.
>
> This is cheap to explain, symmetric, rewards partial lines in proportion to how close they are to completion and ignores dead windows. One call costs about 19.5 microseconds on 7×6 (measured).

4.2 Mathematical derivation of the number of nodes

Let the root be at depth 0 and let every position have b legal moves (empty board, no early end). Level i holds b^i positions, so the number of positions examined for lookahead depth d is

$$N(d) = 1 + b + b^2 + \ldots + b\text{^}d = (b^{d+1} - 1) / (b - 1) \in \Theta(b\text{^}d)$$

The positions at depth d are the **leaves**: L(d) = b^d, and every leaf calls the evaluation function once, so evaluation calls grow as b^d. The fraction of nodes that are leaves is b^d / N(d) = (b−1)(b^d)/(b^{d+1}−1) → (b−1)/b: **86 % for b = 7, 80 % for b = 5**. For the recursion: T(d) = b·T(d−1) + Θ(1), T(0) = Θ(1), which by iteration gives the same geometric sum. Each extra ply multiplies the cost by b. On the empty board no column fills and no line of four is possible before ply 7, so for d ≤ 6 on 7 rows or d ≤ 4 on 5 rows the counts must match exactly.

4.3 Predictions (written before implementing)

N1–N5 as above (node and leaf formulas, ≈(b−1)/b leaves, exactness for small d, effective branching slightly below W mid-game). **T1:** 10–60 µs per node; 1 s limit: 7×6 depth 5 fits, depth 6 does not; 6×5 depth 5 fits, 6 borderline; 5×5 depth 6 fits, depth 7 (97,000 nodes) does not. **Q1:** depth 1 beats a random player in more than 70 % of 20 games, depth ≥ 3 in at least 95 %. **Q2:** each extra ply multiplies time by 5–7; quality gains flatten after depth 3–4.

4.4 Results

Empty-board growth: measured against the formula

Board	d	Nodes measured	Formula N(d)	Leaf evaluations	b^d	Time (s)	µs/node	Exact?
5×5	4	781	781	625	625	0.006	7.5	yes

Board	d	Nodes measured	Formula N(d)	Leaf evaluations	b^d	Time (s)	µs/node	Exact?
5×5	6	19,526	19,531	15,620	15,625	0.153	7.8	no (early wins, full columns)
5×5	8	478,366	488,281	378,272	390,625	3.597	7.5	no
6×5	4	1,555	1,555	1,296	1,296	0.015	9.7	yes
6×5	6	55,981	55,987	46,650	46,656	0.569	10.2	no
6×5	8	1,980,451	2,015,539	1,632,000	1,679,616	19.081	9.6	no
7×6	2	57	57	49	49	0.001	13.7	yes
7×6	4	2,801	2,801	2,401	2,401	0.037	13.1	yes
7×6	5	19,608	19,608	16,807	16,807	0.255	13.0	yes
7×6	6	137,257	137,257	117,649	117,649	2.210	16.1	yes
7×6	7	960,793	960,800	810,504	823,543	14.588	15.2	no

Table 4.1 Selected rows of the full table (Appendix). Where the count differs it is below the formula, because wins end branches and columns fill.

Mid-game positions (10 random positions, 6–10 plies)

Board	d = 3 nodes	d = 4 nodes	d = 5 nodes	d = 6 nodes	b_eff at d = 6	Mean time at d = 6 (s)
5×5	143	669	3,118	13,834	4.90	0.108
6×5	246	1,389	7,952	43,241	5.92	0.370
7×6	388	2,625	18,068	120,530	7.03	2.009

Move quality: minimax against a random player (20 games, 10 as first player)

Board	d = 1	d = 2	d = 3	d = 4	d = 5	Time per game at d = 5 (s)
5×5	20-0-0	19-0-1	20-0-0	19-1-0	19-0-1	0.15
6×5	20-0-0	20-0-0	20-0-0	19-0-1	20-0-0	0.48
7×6	20-0-0	20-0-0	20-0-0	20-0-0	20-0-0	1.76

Table 4.3 Wins-draws-losses for the maximiser (random opponent seeded 0–19).

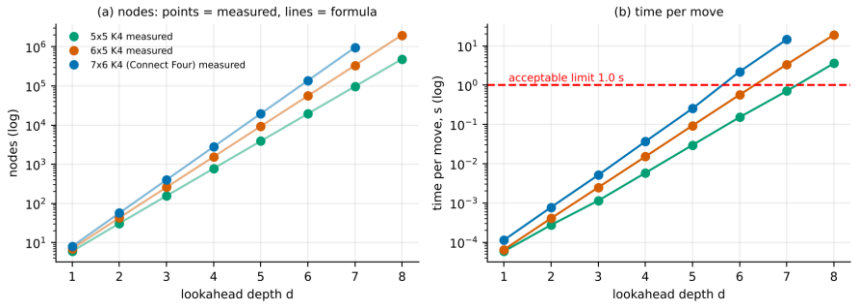


Figure 4.1 (a) nodes against depth: points are measured, lines the formula; (b) time per move against the 1.0 s acceptability limit.

4.5 Comparison of theory and experiment

- **Exactness (N1, N2, N4).** Nodes and evaluation calls match the formulas exactly for d ≤ 6 on 7×6 (137,257 nodes) and d ≤ 5 on the 5-row boards. My claim of exactness for d ≤ 6 **failed for the 5-row boards**: a column of height 5 can already be filled in six plies, so the count is slightly lower (19,526 against 19,531). The lower counts also come from forced wins ending branches (N5).
- **Effective branching (N5).** Mid-game b_eff is below W and falls with depth (7×6: 8.0 at d = 1 to 7.03 at d = 6) as columns fill and lines are completed.

- **Time (T1).** The per-node time was **lower than predicted**: 7–10 µs on the small boards and 13–16 µs on 7×6, not 10–60 µs, because the evaluation scans a short list of precomputed windows. Time per node rises a little with the board size (more windows).
- **Quality (Q1, Q2).** Depth 1 already wins 100 % against the random player (above the predicted 70 %), so the random opponent is too weak to measure how quality improves with depth; the two losses at depth 2 and 4 are single games lost to chance. This part of the evidence is therefore weak and I do not rely on it for the choice of depth.

4.6 Scalability inquiry

Acceptability criterion: a decision must be ready in **at most 1.0 s** per move, an interactive turn-based game. **Practical limit** from the data (empty or mid-game boards, Table 4.1 and Figure 4.1b): 7×6 depth 5 (0.26 s) fits and depth 6 (2.2 s) does not; 6×5 depth 6 (0.57 s) fits and depth 7 (3.4 s) does not; 5×5 depth 7 (0.71 s) fits and depth 8 (3.6 s) does not. In my prediction I had put the 6×5 and 5×5 limits one ply lower, because the per-node time was lower than I estimated. Plain minimax therefore becomes impractical at about 10^5 nodes (depth 6 on 7×6) in Python, and every extra ply costs 5–7× more.

Where the inefficiency originates (measured in Task 7 diagnostics): (1) about 86 % of all nodes are leaves and 82 % of the run time is spent in the evaluation function (19.5 µs × calls); (2) the same position is reached by many move orders: at depth 6 from the empty board only 16,422 of the 117,649 sequences are distinct positions; (3) a branch is searched in full even when an earlier alternative has already shown that it cannot change the decision; (4) the moves are tried left to right, which is unrelated to their quality. A redesign must therefore **evaluate fewer leaves** (pruning by bounds), **not recompute the same position**, **search promising moves first** so that pruning triggers early, and, if possible, make the evaluation cheaper. I implement this in Task 7.

Task 5 – Deriving a Constraint-Propagation Algorithm from a Principle (7 marks)

5.1 Starting point and representation

> **Given principle**
> A value may be removed from a variable's domain if some neighbouring variable has no value that is compatible with it.

Variables and domains. Each exam i is a variable with a domain D(i), the set of slots still possible; the domains are stored as Python sets so that a value can be removed in constant time. **Constraints** are binary and stored **per arc (directed pair)**: `rel[(i,j)]` is a predicate f(a, b) that is true when slot a for exam i and slot b for exam j are compatible. A conflict (shared students) is a ≠ b in both directions; a precedence 'i before j' is a < b on arc (i,j) and a > b on arc (j,i); a unary restriction (hall booked) simply shrinks the starting domain. `nbrs[i]` lists the exams that share a constraint with i. Storing the constraint on each arc, not on each pair, is what allows a value to be examined from the viewpoint of one variable at a time, exactly as the principle is phrased.

5.2 Deriving the procedure

(a) How an unsupported value is detected. Take an arc (i, j) and a value a ∈ D(i). The principle says a is removable when no b ∈ D(j) satisfies rel(a, b). So REVISE(i, j) loops over a ∈ D(i), searches D(j) for a supporting b (stopping at the first one found) and deletes a if the search fails. Each compatibility test is counted as one constraint check.

(b) Which pairs must be re-examined after a removal. Removing a value from D(i) can only hurt variables whose values were *supported by it*: the neighbours k of i. So the arcs (k, i) must be examined again. Arc (i, j) does not need re-checking when the removal was made because of j: the removed value a had no compatible b in D(j), so no value of D(j) can have lost its support through a. This gives the queue rule: after D(i) shrinks while revising (i, j), add every arc (k, i) with k ≠ j that is not already queued.

(c) Failure. If D(i) becomes empty (a **domain wipe-out**), no value of i is consistent with the others, so the problem (or the current branch in search) has no solution. **(d) Fixed point.** Initially all arcs are queued; the process stops when the queue is empty, i.e. when every value of every domain has a support on every arc (arc consistency). It terminates because every re-queue is caused by a removal and there are finitely many values.

(e) Combination with backtracking (MAC = maintaining arc consistency). Propagate once at the root. Then choose an unassigned exam, try each value in its current domain (assignment = domain reduced to one value), propagate the arcs (k, i) of the assigned exam on a copy of the domains, recurse if no wipe-out occurred, otherwise try the next value. The copy makes backtracking trivial. Two variable orders are compared: static (index order) and **MRV** (fewest remaining values, ties broken by most constraints).

```
function PROPAGATE(csp, dom, queue):           // returns False on a domain wipe-out
    inq = set(queue)
    while queue not empty:
        (i, j) = pop front(queue);  inq.remove((i, j))
        if REVISE(i, j):                                  // D(i) lost at least one value
            if D(i) is empty:  wipeouts += 1;  return False
            for each k in nbrs[i], k != j:
                if (k, i) not in inq:  append (k, i) to queue;  inq.add((k, i))
    return True
```

```
function REVISE(i, j):
    removed = {}
    for a in D(i):
        supported = False
        for b in D(j):
            checks += 1
            if rel(i, j)(a, b):  supported = True;  break
        if not supported:  removed.add(a)
    D(i) = D(i) - removed
    return removed is not empty

function MAC(dom, assigned):
    if len(assigned) == n:  return solution
    i = choose unassigned exam              // static order, or MRV: min |D(i)|, ties: max degree
    for a in sorted(D(i)):
        nodes += 1;   nd = copy of dom;   nd[i] = {a}
        if PROPAGATE(csp, nd, [(k, i) for k in nbrs[i]]):
            r = MAC(nd, assigned + {i: a});   if r: return r
        backtracks += 1
    return failure
```

Complexity of the propagation: with e arcs and d slots each REVISE costs at most d^2 checks and an arc is re-queued at most d times (once per removed value), so the worst case is $O(e \cdot d^3)$; with d = 3–8 slots, as here, propagation is cheap compared with search.

5.3 Predictions (written before implementation)

No.	Prediction
I1	Instance A (precedence chain, conflicts, one unary restriction): propagation alone shrinks every domain to one value: solved with 0 search nodes, 0 wipe-outs.
I2	Instance B (A fixed to slot 0; B, C in {0,1}; A, B, C pairwise in conflict): propagation alone detects failure with 0 assignments, exactly 1 wipe-out.
I3	Pigeonhole (k+1 mutually conflicting exams, k slots): propagation sees nothing at the root; BT and MAC both need about k! assignments; MAC visits fewer nodes by a factor 2–6.
I4	Hidden core (12 loose exams first, an unsatisfiable 5-exam clique with 4 slots last): BT and static-order MAC thrash (hit the 100,000-node cap); MAC with MRV/degree ordering finishes in < 1,000 nodes.
I5	Random timetables (3 slots, mean degree 4.2): BT nodes grow roughly exponentially with n; MAC cuts nodes by 1–2 orders of magnitude at n = 50–60 but spends more checks per node, so the saving in checks is smaller than in nodes.
I6	No value that belongs to a solution is ever removed (argued by reasoning in 5.7, sanity-checked by brute force).

5.4 Test instances and visualisation of shrinking domains

Instance A (8 exams, 4 slots 0–3): Maths is restricted to slot 0 (hall booking); syllabus order Maths < Physics < Chemistry < Biology; conflicts English–{Maths, Physics, Chem}, History–{Biology, English, Physics, Chem}, Art–{History, Physics, Chem}, Music–{Maths, History, Art}. **Result:** propagation alone made 59 revisions, removed 20 values, used 151 constraint checks, 0 wipe-outs, and reduced seven exams to a single slot, but **Music kept {1, 2}**, so my prediction I1 (every domain a singleton) was **not met**: nothing forces Music into one of its two remaining slots. The assignment still verified against all constraints (backtracking with 8 nodes, 0 backtracks). I therefore refined the instance to **A2** by adding one conflict (Music–Physics): 64 revisions, 21 removals, 161 checks, 0 wipe-outs and **all domains singletons** (Maths 0, Physics 1, Chemistry 2, Biology 3, English 3, History 0, Art 3, Music 2), verified against all constraints, with 0 search nodes.

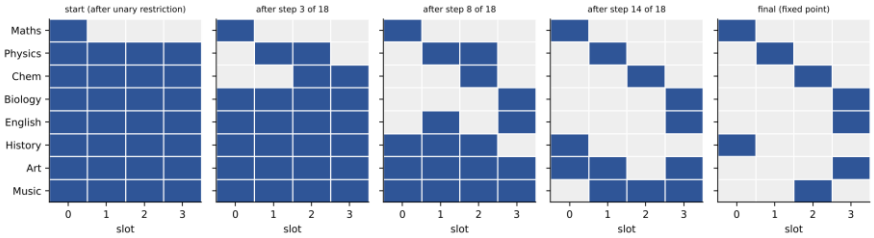


Figure 5.1 Instance A2: shrinking domains (dark = slot still possible) after selected propagation steps. The chain Maths < Physics < Chem < Biology collapses first; the conflicts then fix the remaining exams.

Instance B (4 exams, 3 slots): A fixed to slot 0, B and C in {0, 1}, A, B, C and D pairwise in conflict. Propagation: revise B against A removes 0 from D(B) → {1}; C against A → {1}; B against C then empties D(B): **1 wipe-out after 5 revisions, 3 removals and 9 checks, with 0 assignments made** (MAC: 0 nodes; plain BT needs 2 nodes and 2 backtracks to find the same failure). I2 confirmed.

5.5 Instances designed to challenge the algorithm

Pigeonhole: k+1 mutually conflicting exams, k slots

k	Exams	BT nodes	MAC nodes	BT ÷ MAC	BT checks	MAC checks	k!
4	5	64	40	1.6	556	944	24
5	6	325	205	1.6	4,405	5,605	120
6	7	1,956	1,236	1.6	38,166	37,800	720
7	8	13,699	8,659	1.6	363,055	291,305	5,040
8	9	100,000 (cap)	69,280	≥ 1.4	3,449,882	2,539,360	40,320
9	10	100,000 (cap)	100,000 (cap)	-	4,349,818	3,967,032	362,880

Root propagation removes nothing (every slot has a support in every other exam), so it cannot detect the failure; both algorithms must try the k! assignments. MAC needs **1.6× fewer nodes** (I predicted 2–6×, a miss in magnitude) and from k = 6 also fewer checks, but the growth is factorial for both (each extra exam multiplies the nodes by about 5, 6 and 7). This is the classic limit of arc consistency: it checks pairs, not the global 'k+1 values in k slots' argument.

Hidden unsatisfiable core

Algorithm	Nodes	Backtracks	Checks	Wipe-outs	Result
BT	100,000	99,986	852,508	36,560	node cap
MAC (static order)	100,000	99,987	2,033,613	57,634	node cap
MAC + MRV	100,000	99,989	2,039,118	57,825	node cap

I predicted (I4) that MRV ordering would reach the core first and finish in under 1,000 nodes. On the reported instance (seed 5) it did **not**. I therefore ran 20 seeds: MAC + MRV finished in 40 nodes in **14 of 20**, BT and static MAC in 0 of 20. The six failures (seeds 4, 5, 6, 7, 9, 14) are those where one of the loosely connected exams has degree ≥ 4, which ties or beats the degree of the clique members (4) in the MRV tie-break, so the search starts in the wrong region. The rescue by variable ordering is therefore real but **fragile**: it depends on a heuristic tie-break, not on propagation itself.

5.6 Computational growth with the number of exams

n	BT nodes	MAC nodes	MAC+MRV nodes	BT checks	MAC checks	MAC+MRV checks	Solved / unsat / capped (BT)	(MAC)
10	110	18	9	603	611	503	7 / 13 / 0	7 / 13 / 0
20	5,851	56	14	32,641	2,950	1,030	7 / 13 / 0	7 / 13 / 0
30	49,541	295	25	281,833	7,183	1,602	12 / 2 / 6	12 / 8 / 0
40	86,310	243	39	513,305	21,650	2,721	2 / 1 / 17	8 / 12 / 0
50	95,080	920	49	556,215	98,275	4,165	1 / 0 / 19	9 / 11 / 0
60	100,000	560	69	602,065	43,934	4,362	0 / 0 / 20	12 / 8 / 0

Table 5.3 Mean per instance over 20 random timetables per n (seeds 0–19; 3 slots, mean degree 4.2, node cap 100,000). Capped BT runs are censored, so the BT averages are lower bounds.

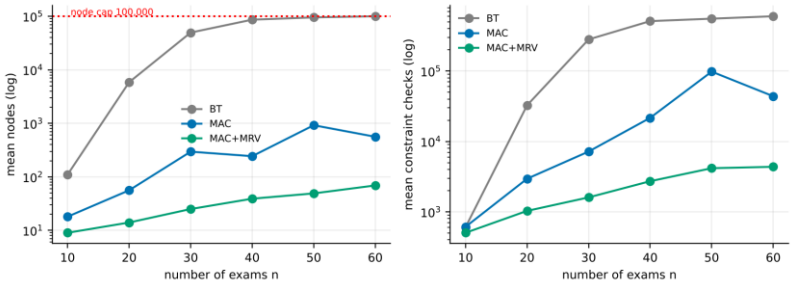


Figure 5.2 Mean nodes and constraint checks against the number of exams (log scale).

BT grows ×53 from n = 10 to 20 and is capped from n = 30 onward (0 of 20 instances solved at n = 60), while MAC grows from 18 to 560 nodes and MAC + MRV from 9 to 69. At n = 10 MAC needs 6× fewer nodes but the **same** number of checks (611 against 603): the per-node propagation cost cancels the node saving, as predicted (I5). At n = 60 MAC + MRV needs 4,362 checks, 138× fewer than BT's lower bound of 602,065. Also visible: BT cannot even tell unsatisfiable from hard (all 20 capped at n = 60) while MAC decides every instance (12 solvable, 8 not).

5.7 Correctness inquiry: why removing an unsupported value never removes a value that belongs to a solution

Claim. Let a ∈ D(Xi) and let Xj be a neighbour with no b ∈ D(Xj) such that (a, b) is allowed by the constraint between Xi and Xj. Then no solution of the problem assigns Xi = a.

Reasoning. We maintain the invariant **(I): for every solution s of the original problem, s(Xv) ∈ D(Xv) for every variable Xv.** At the start (I) holds, because the domains are the original ones (a unary restriction removes only values that are forbidden anyway). Suppose (I) holds and the procedure removes a from D(Xi) because Xj has no compatible value in D(Xj). Assume, for contradiction, that some solution s has s(Xi) = a. A solution satisfies every constraint, so the pair (a, s(Xj)) is allowed. By (I), s(Xj) ∈ D(Xj). Therefore s(Xj) is a value of D(Xj) compatible with a, which contradicts ‘no compatible value exists in D(Xj)’. Hence no solution uses a, and deleting it keeps (I) true. By induction on the number of removals, (I) holds when the procedure stops.

Consequences. (1) A wipe-out (empty D(Xi)) together with (I) means a solution would need s(Xi) ∈ ∅, so there is no solution: failure detection is sound. (2) In MAC, an assignment Xi = a is just the original problem

with D(Xi) reduced to {a}; the same argument shows that propagation inside that branch never removes a value belonging to a solution *of that branch*, so no solution is lost and a backtracking search over the remaining branches stays complete. (3) The converse does not hold: arc consistency is only a necessary condition, which is why the pigeonhole instance passes propagation yet has no solution. The sanity check (300 random small instances, 1,924 removals, 0 removed values belonging to any solution) agrees with the argument but is not the proof.

Task 6 – Exhaustive Search, Local Search and Combinatorial Growth (8 marks)

6.1 Design of the exhaustive strategy

- **Representation.** A tour is a permutation of the customers 1 … n−1, the depot 0 is fixed at both ends. Fixing the depot removes the n rotations of the same cycle, so the number of candidate tours is (n−1)!. (Reversed tours are counted separately; halving would give (n−1)!/2 without changing the growth.) Distances are Euclidean between random points in a 100 × 100 square (instance seeds reported) and stored in an n × n matrix.
- **Generation.** Recursive extension: a partial tour is extended by every unvisited customer (a 'used' array marks visited ones), so each of the (n−1)! permutations is generated exactly once.
- **Evaluation.** The length is accumulated incrementally (add one leg per extension); at a complete tour the return leg to the depot is added.
- **Retention.** The cheapest complete tour and its length are kept in `best`, replaced when a shorter one is found.

```
function EXHAUSTIVE(d):                          // d = n x n distance matrix, depot = 0
    best = (infinity, none);  path = [0];  used = {0}
    function EXTEND(last, cost, depth):
        nodes += 1
        if depth == n-1:                          // all customers visited
            total = cost + d[last][0];  tours += 1
            if total < best.cost:  best = (total, path + [0])
            return
        for c in 1 .. n-1 not in used:
            used.add(c);  path.append(c)
            EXTEND(c, cost + d[last][c], depth + 1)
            path.pop();  used.remove(c)
    EXTEND(0, 0, 0)
    return best
```

6.2 Correctness check

Instance whose optimum is known independently: 8 points on a circle of radius 50 (depot included). The optimal tour is the polygon visiting the points in angular order, whose length is the perimeter. Result: exhaustive cost = 293.941316 = perimeter 293.941316 (difference < 10^{-9}), and the returned tour visits the cities in angular order. In addition, the exhaustive result was compared with `itertools.permutations` brute force on 30 random instances (n = 6–8) and with the Held–Karp dynamic-programming optimum on 10 random instances (n = 8–10): **0 disagreements** in 40 comparisons.

6.3 Growth: prediction and validation

Prediction (before experimenting). Candidate tours = (n−1)!; adding one customer multiplies it by n−1 (about 9–10 for n = 10–11). The search tree has $\Sigma_{k=0}^{n-1}$ (n−1)!/(n−1−k)! ≈ e·(n−1)! nodes, so time(11)/time(10) should be about 10 (range 8–13). I also predicted 1–3 µs per node (n = 10 about 1–3 s, n = 11 about 10–30 s).

n	Cust.	Tours counted	(n−1)!	Equal	Tree nodes	Time (s)	Ratio to previous
5	4	24	24	yes	65	0.000	-
6	5	120	120	yes	326	0.000	2.2×
7	6	720	720	yes	1,957	0.000	6.5×
8	7	5,040	5,040	yes	13,700	0.003	7.8×

n	Cust.	Tours counted	(n−1)!	Equal	Tree nodes	Time (s)	Ratio to previous
9	8	40,320	40,320	yes	109,601	0.027	7.9×
10	9	362,880	362,880	yes	986,410	0.280	10.5×
11	10	3,628,800	3,628,800	yes	9,864,101	2.811	10.0×
12	11	39,916,800	39,916,800	yes	108,505,112	35.002	12.5×

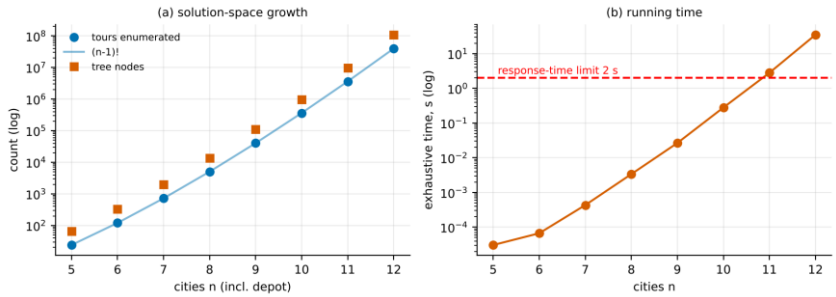


Figure 6.1 (a) tours enumerated and tree nodes against n (log scale); (b) exhaustive running time against the 2 s response-time limit.

Comparison. The tour counts equal (n−1)! exactly for all eight sizes (G1 confirmed). Tree nodes are 2.7× the tour count (e·(n−1)! → 2.72) and the time ratio per added customer rises from 6.5× to 12.5× towards n−1 (G2 confirmed; the n = 12 ratio of 12.5 is at the top of my range). **Miss:** the time per node was **0.27–0.32 µs, three to ten times faster than predicted**, so n = 10 took 0.28 s (predicted 1–3 s) and n = 11 took 2.8 s (predicted 10–30 s). The shape of the prediction was right, the constant was wrong; this changes where the deployment boundary lies by about one customer.

Extrapolation at the measured rate (3.5 million nodes per second): n = 13: 371 s (0.10 h); n = 15: 18.8 h; n = 20: 2,990 years; n = 25: 1.5×10^{10} years.

6.4 Local-search alternative

Design: 2-opt hill climbing with random restarts. A tour is a permutation. A neighbour is made by reversing a segment of the tour (a 2-opt move, which replaces two edges by two others). The search takes the first improving move it finds and repeats until no segment reversal shortens the tour (a local optimum). A **restart** begins again from a new random permutation; the result of the *R-restart* variant is the best of R climbs (here R = 10). Per pass there are about $n^2/2$ neighbours, so one climb costs a polynomial number of evaluations.

```
function HILL_CLIMB(d, restarts):
    best = infinity
    repeat restarts times:
        tour = random permutation of 1..n-1, depot at both ends
        improved = True
        while improved:
            improved = False
            for i in 1 .. n-2:  for j in i+1 .. n-1:
                new = tour with segment tour[i..j] reversed
                evals += 1
                if length(new) < length(tour):  tour = new;  improved = True
        best = min(best, length(tour))
    return best
```

Prediction. L1: from one random start 2-opt gets stuck: mean excess 2–8 % over the optimum and the optimum reached in only 20–60 % of starts (n = 8–11). L2: with 10 restarts the mean excess falls below 1 % and the optimum is reached in most of the 20 runs (≥ 60 %). L3: n = 100 solved in about 1 s.

n	Instance	Optimum	Single start: mean % above	max %	Reached optimum	ms/run	10 restarts: mean %	max %	Reached optimum	ms/run
8	1	318.61	0.16	0.45	13/20	0.02	0.00	0.00	20/20	0.11
8	2	281.04	0.00	0.00	20/20	0.01	0.00	0.00	20/20	0.12
8	3	271.79	0.00	0.00	20/20	0.01	0.00	0.00	20/20	0.12
10	1	344.77	2.31	4.44	3/20	0.02	0.01	0.08	18/20	0.18
10	2	301.11	0.38	2.57	17/20	0.02	0.00	0.00	20/20	0.19
10	3	304.80	0.00	0.00	20/20	0.02	0.00	0.00	20/20	0.21
11	1	348.69	2.01	3.91	9/20	0.02	0.00	0.00	20/20	0.23
11	2	310.31	0.37	2.49	17/20	0.02	0.00	0.00	20/20	0.19
11	3	310.00	0.17	0.69	15/20	0.02	0.00	0.00	20/20	0.21
12	1	362.67	1.45	3.32	6/20	0.02	0.08	1.58	19/20	0.23

Table 6.2 20 independent runs per instance (seeds 1000·instance + run). Optimum from exhaustive search. Instance seeds 1–3 (n = 12: seed 1 only, exhaustive 35 s).

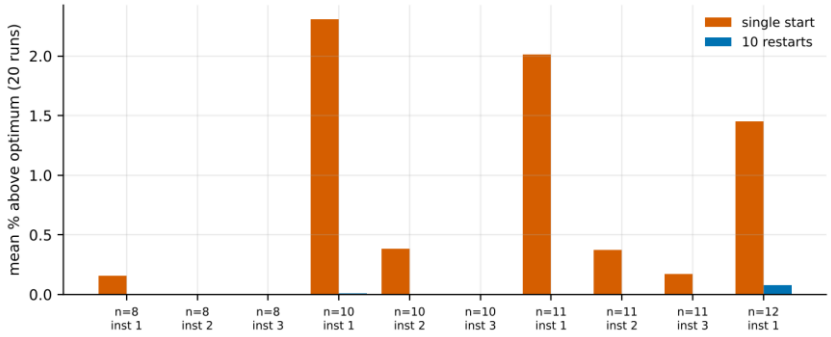


Figure 6.2 Mean percentage above the exhaustive optimum, single start against 10 restarts.

Local-minima study (n = 10, instance 1, 2,000 independent single-start climbs): only **4 distinct local optima** were reached, lengths 344.77 (the global optimum, 28.8 % of starts), 345.03 (+0.08 %, 21.1 %), 346.45 (+0.49 %, 19.6 %) and 360.09 (**+4.44 %**, 30.6 %). So from one start the climb ended in a trap in 71 % of the cases.

Larger sizes (no exhaustive optimum is possible): nearest-neighbour baseline against 2-opt with 10 restarts (20 runs, seed-7 instances):

n	Nearest neighbour	2-opt mean	best	worst	Gain over NN	Time per run	2-opt evaluations
20	395.6	355.6	355.0	365.5	10.1 %	0.001 s	5,891
30	538.9	427.0	426.3	433.6	20.8 %	0.002 s	16,768
50	601.2	528.4	516.4	548.3	12.1 %	0.006 s	52,097
100	1,078.2	858.8	845.5	875.9	20.3 %	0.029 s	264,865

Comparison. Time grows polynomially (evaluations ×2.8 from n = 20 to 30, ×3.1 to 50 and ×5.1 to 100, about $n^{2.4}$ overall), and n = 100 takes 0.03 s, **30× faster than I predicted**, while exhaustive search would need 99! tours. **L1 was too pessimistic in quality:** the single-start mean excess was only 0–2.3 % (the worst instances 1–2 %), though the hit rate (3/20 to 20/20, 70 % overall) shows the traps are real; the high end of my 20–60 % hit-rate range is consistent for the harder instances. **L2 confirmed:** with 10 restarts the mean excess is at most 0.08 % and the optimum was reached in 197 of 200 runs (98.5 %); the only single run that missed by more than 0.1 % was n = 12 at +1.58 %. Restarts cost 10× the time (0.1–0.2 ms) and removed nearly all of the error.

6.5 Inquiry

Why a few more customers cause a dramatic rise in effort. The solution space is the set of permutations, which grows by a factor n−1 with every added customer: going from 10 to 12 locations multiplies the tours by 10 × 11 = 110 (measured time: 0.28 s to 35 s, ×125). Each new customer multiplies the existing space instead of adding to it, and nothing in plain enumeration can skip a tour, because the length of a tour is only known when it is complete. A computer 1000 times faster would gain only about three customers.

Why local search can fail to reach the optimum. A 2-opt climb stops where no single segment reversal helps, which is a **local minimum** of the length surface: the optimum may need a sequence of moves, some of which first lengthen the tour, to be reached. The study above found one such trap at +4.44 % that captured 30.6 % of the starts. **Plateaus** (several neighbours of equal length, where the search cannot tell which way to move) occur when distances are tied, for example with integer grid distances; my random Euclidean instances had no exact ties, so I observed local minima but not plateaus. Random restarts, accepting worse moves (simulated annealing) or larger neighbourhoods are the standard escapes, and restarts alone reduced the failures to 3 of 200 runs.

6.6 Deployment decision

Response-time limit chosen: 2.0 seconds, the time an interactive dispatcher screen can take before the user perceives the system as stalled; batch planning at night would allow more, and the same method would simply move the boundary. **Exhaustive search** is deployed for **n ≤ 10 (nine customers plus depot): 362,880 tours in 0.28 s**, a margin of 7× against the limit that also covers a machine 2–3× slower. n = 11 needs 2.8 s, which is already over the limit on my hardware, and n = 12 needs 35 s. **From n ≥ 11 the planner switches to 2-opt with 10 restarts**, which found the optimum in 79 of 80 runs at n = 11–12 within 0.25 ms and scales to n = 100 in 0.03 s. In this regime the quality guarantee is lost (no certificate of optimality), so for n = 11–12 where the exhaustive result is just affordable I would run local search first and verify it exhaustively in the background; this is a suggestion, not tested here.

Prediction check (D1): I predicted ‘exhaustive up to 10 customers+depot, switch at n ≥ 11’; the measurement agrees (the boundary falls between n = 10 at 0.28 s and n = 11 at 2.81 s), although for the wrong reason: my per-node speed was off by a factor of about 5, and my predicted times of 1–3 s (n = 10) and 10–30 s (n = 11) would also have put the boundary at n = 10.

Task 7 – Day-4 Design Challenge: Diagnose, Redesign and Validate (13 marks)

Solution selected: the plain depth-limited minimax of Task 4 (Connect-Four, 7 × 6). Review of Tasks 1–6 shows three solutions with a clear computational limit: exhaustive TSP (Task 6; (n−1)! cannot be reduced without changing the problem), backtracking without propagation (Task 5; its limits were shown mainly by thrashing) and plain minimax (Task 4). Minimax was chosen because its cost is large, its source is identifiable and a redesign can be validated exactly (the root value must be identical).

A. Diagnose the computational bottleneck (2 marks)

The questions are answered with measurements on the Task 4 solution; the predictions of the hypotheses were written before measuring (`t7_predictions.txt`).

Question	Evidence from my own experiments	Conclusion
Where is most effort spent?	Task 4: leaves are 86 % of nodes on 7 columns (b^d / N(d) → (b−1)/b). Diagnostic D2 (10 mid-game positions, depth 4): one evaluation call = 19.5 µs; 20,395 calls = 0.398 s of 0.485 s total, i.e. **82 %** of the run time is the evaluation function (hypothesis H1: ≥ 70 %).	The leaf evaluations dominate. The cheapest win is to evaluate fewer leaves.
Which operation grows significantly?	Nodes and evaluation calls grow as b^d: 2,801 → 19,608 → 137,257 → 960,793 nodes for d = 4 → 7 on 7×6, time 0.037 → 14.6 s.	Evaluation calls multiply by 7 per ply.
Is work repeated?	D1: distinct positions against move sequences from the empty board: d = 3: 238 of 343 (69 %); d = 4: 1,120 of 2,401 (47 %); d = 5: 4,263 of 16,807 (25 %); d = 6: 16,422 of 117,649 (**14 %**); d = 7: 54,131 of 823,543 (7 %).	At depth 6, 86 % of the sequences re-reach a position already searched (H2 predicted 20–50 % distinct; the real value 14 % is lower, i.e. transpositions are more common than I predicted).
Which property of the algorithm causes it?	Plain minimax (i) searches every branch to full depth even when an earlier alternative already makes it irrelevant (Task 3: 0 cutoffs without pruning), (ii) searches columns left to right, unrelated to move quality, (iii) stores nothing, so every transposition is recomputed.	No use of bounds, no ordering, no memory of results.

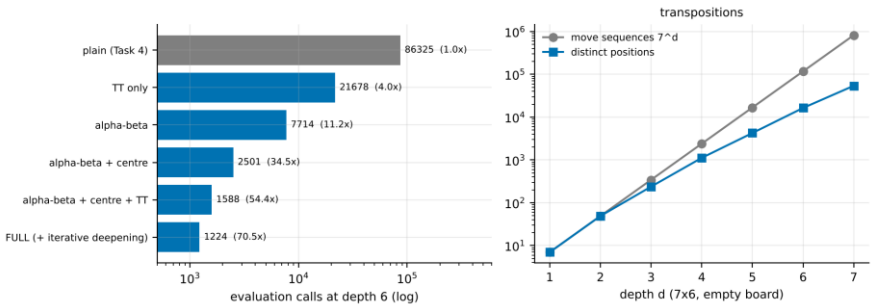


Figure 7.1 Right: move sequences against distinct positions (transpositions). Left: evaluation calls at depth 6 for each design variant (mean of 10 mid-game positions; the number in brackets is the reduction against plain minimax).

B. Design objective (2 marks)

No.	Measurable objective (7×6, 10 mid-game positions of 8–14 plies, evaluation function unchanged)
O1	**Correctness:** the root value of the redesign equals the root value of plain minimax at the same depth, with zero mismatches in all tests (and the chosen move is one of the optimal moves).
O2	**Work:** at fixed depth 6, at least **10× fewer evaluation calls** than plain minimax, counting all the work of every iteration.
O3	**Depth within the Task 4 time limit:** deepest completed search within 1.0 s per move at least 7 (plain minimax: 5).

No.	Measurable objective (7×6, 10 mid-game positions of 8–14 plies, evaluation function unchanged)
O4	**Memory:** transposition-table memory below 100 MB at depth 7.

C. The revised computational strategy (4 marks)

The redesign does not replace minimax by a different problem; it keeps the same game tree, the same evaluation function and the same depth, and exploits four properties of the problem to avoid work whose result cannot affect the decision.

Component	Property exploited	Operations avoided	New cost / trade-off
C1 Alpha-beta bounds	A node whose value already exceeds what the opponent will allow (≥ β) or falls below what I already have (≤ α) cannot change the root decision.	Whole sub-trees and their evaluation calls.	Only the best root move gets an exact value; the others return bounds.
C2 Centre-first move ordering	On Connect-K, central columns take part in more lines, so they are usually the strongest moves: a good move searched first gives the strongest bound early (Task 3: the best-first order gives ≈ √ of the work).	Cutoffs fire at the first or second child instead of the last.	Sorting cost per node; the ordering is domain knowledge, not general.
C3 Transposition table (Zobrist key; EXACT / LOWER / UPPER flags)	The value of a position depends only on the position and the remaining depth, not on the order of the moves that led to it.	Re-searching a position reached by another move order (86 % of the sequences at depth 6).	Memory (measured 125–142 bytes per entry in Python), hash update per move, a 64-bit key can in principle collide (probability about $n^2/2^{65} \approx 10^{-10}$ for 10^5 entries).
C4 Iterative deepening with the previous best move first	The best move at depth d−1 is usually the best (or close) at depth d.	Wasted effort in the first, poorly ordered, subtrees of the deepest search; also gives an anytime result at any time limit.	Repeats the shallow iterations (their cost is geometrically small, 1/b of the next).

Design rule that keeps the result identical to minimax (soundness)

A win scores ±(1,000,000 + remaining depth), so a stored value is only valid for the **same remaining depth**. The table therefore returns a value only when the stored depth equals the required depth: an EXACT entry is returned, a LOWER or UPPER entry narrows α or β (and may cause a cut). Entries from other depths are used **only for move ordering** (their best move is tried first). This costs some reuse across iterations but guarantees O1. Flags are set by comparing the result with the original window: value ≤ α_o → UPPER, value ≥ β_o → LOWER, otherwise EXACT, where α_o and β_o are the bounds before the table narrowed them.

```
function DECIDE(position, me, max_depth, time_limit):
    tt = {}                                                    // transposition table, key = 64-bit Zobrist
hash
    for depth = 1, 2, 3, ...:                                  // C4: iterative deepening
        move, value = ROOT(position, depth)                    // stops when the time limit is reached
    return the move of the last COMPLETED depth


function ROOT(pos, depth):
    moves = centre-first order, with the stored best move of pos moved to the front       // C2 + C4
    best = -inf
    for m in moves:  v = SEARCH(play(pos, m), depth-1, alpha = best, beta = +inf);  keep the best (move, v)
    tt[pos.key] = (depth, EXACT, best, best move);  return best move, best


function SEARCH(pos, depth, alpha, beta):
    nodes += 1
    if last move won:  return +/-(1000000 + depth);   if board full: return 0
    if depth == 0:  evals += 1;  return EVALUATE(pos)            // the unchanged Task 4 evaluation function
    alpha0, beta0 = alpha, beta
    e = tt.get(pos.key)                                        // C3
    if e exists:
        ordering_move = e.best_move
        if e.depth == depth:                                    // same remaining depth: value is valid
```

```
            if e.flag == EXACT:  return e.value
            if e.flag == LOWER: alpha = max(alpha, e.value)  else if e.flag == UPPER: beta = min(beta,
e.value)
            if alpha >= beta:  return e.value
    moves = centre-first order of the legal columns, ordering_move first
    best = -inf (MAX node) or +inf (MIN node)
    for m in moves:
        v = SEARCH(play(pos, m), depth-1, alpha, beta)
        MAX node: best = max(best, v);  alpha = max(alpha, best)      MIN node: best = min(best, v);  beta =
min(beta, best)
        if alpha >= beta:  break                                    // C1: cutoff
    flag = UPPER if best <= alpha0  else LOWER if best >= beta0  else EXACT
    tt[pos.key] = (depth, flag, best, best move);  return best
```

Extra memory and preprocessing: a Zobrist table of random 64-bit numbers, one per (cell, player) (126 numbers on 7×6), made once; an incrementally updated key (one XOR per move and per undo); the transposition table (cleared for every move decision). **New trade-offs:** more code (about 130 lines against 30), a memory/time trade-off in the table, and the loss of exact values for non-best root moves.

D. Predictions made before implementing (1 mark)

No.	Prediction (all written before the redesign existed)
P1	Evaluation calls at depth 6 against plain: alpha-beta alone 3–8× fewer; with centre ordering 10–30×; + transposition table a further 1.3–2×; full design (all iterations counted) 20–60×; table alone (no pruning) 1.5–3×.
P2	Ordering is worth more than the table (components are not additive).
P3	Deepest completed depth within 1.0 s on 7×6: plain 5, redesign 7 or 8 (5×5: plain 6, redesign ≥ 9).
P4	Table size at depth 7: 100,000–600,000 entries, 30–150 MB in Python (so O4 probably missed or borderline).
P5	At depth 1–2 the redesign is slower than plain (hash and sort overhead): 1.0–3×. The original stays preferable for depth ≤ 2.
P6	Head-to-head at 0.2 s per move, 20 games: the redesign beats plain minimax (depth 4) in ≥ 14 of 20 games (draws count half).
P7	Zero correctness mismatches.
P8	Alpha-beta returns an exact value only for the best root move; the redesign cannot be used when the exact value of every move is needed.

E. Implementation and validation (2 marks)

The prototype is `t7_improved.py` (Appendix A); each component can be switched off for an ablation. Experiments: `t7_run.py`, `t7_extra.py`; outputs in Appendix B.

E1 Correctness (objective O1)

3,360 tests (position, depth, configuration) on three board sizes (5×5 depth 1–5, 6×5 depth 1–5, 7×6 depth 1–4; 40 positions each of 3–22 plies, including positions with forced wins and blocks) × the six configurations: **value mismatches = 0**, and in every test the chosen move had the optimal minimax value (checked against the exact minimax value of every root move, since several moves can tie). O1 is met.

E2 Ablation: 7×6, 10 mid-game positions (8–14 plies), mean per position

Depth	Configuration	Nodes	Evaluations	Evals reduction	Time (s)	Speed-up	TT entries
6	Plain minimax (Task 4)	106,812	86,325	1.0×	2.178	1.0×	0
6	Transposition table only	30,987	21,678	4.0×	0.563	3.9×	4,504
6	Alpha-beta only	11,239	7,714	11.2×	0.194	11.2×	0
6	Alpha-beta + centre order	4,438	2,501	34.5×	0.066	33.0×	0
6	Alpha-beta + centre + TT	3,054	1,588	54.4×	0.043	50.2×	889
6	**Full (+ iterative deepening)**	2,094	1,224	**70.5×**	0.034	**64.0×**	441

Depth	Configuration	Nodes	Evaluations	Evals reduction	Time (s)	Speed-up	TT entries
5	Plain minimax	16,583	13,090	1.0×	0.317	1.0×	0
5	Full design	1,048	676	19.4×	0.018	17.6×	201
4	Plain minimax	2,462	2,040	1.0×	0.050	1.0×	0
4	Full design	334	212	9.6×	0.006	9.0×	76

Table 7.1 Reduction = plain ÷ configuration. 'Full' counts the work of every iteration (depths 1…d). The time reduction follows the evaluation reduction closely because evaluation dominates the cost.

E3 Depth reached within the 1.0 s budget (objective O3)

Board	Positions	Plain: deepest depth in ≤ 1 s	Redesign: depth completed in 1.0 s	Same value at the plain depth
5×5	5	7, 7, 7, 7, 10 (mean 7.6)	25, 30, 30, 30, 30 (mean 29.0)	5 / 5
6×5	5	6, 6, 6, 6, 6 (mean 6.0)	14, 30, 12, 15, 30 (mean 20.2)	5 / 5
7×6	8	5 on all 8 positions	10, 10, 30, 10, 16, 26, 23, 10 (mean 16.9, median 13)	8 / 8

A depth of 30 is the search limit of the experiment: those positions are decided early (a forced win or loss is found and the tree collapses), so the largest numbers say more about the positions than about the method. The honest, position-independent statement is the 7×6 result of the four positions that used the whole second: **depth 10 for the redesign against depth 5 for plain minimax**, i.e. objective O3 (≥ 7) is met on every position tested.

E4 Memory (objective O4)

Setting (7×6)	Table entries	Peak memory (tracemalloc)	Bytes per entry
Fixed depth 6 (mean of 5)	392	0.1 MB	138
Fixed depth 7	920	0.1 MB (max 0.2)	125
Fixed depth 8	1,763	0.3 MB (max 0.4)	142
5 s budget, depth 10–11 (4 positions)	18,309–23,241	2.7–4.5 MB	about 150

O4 is met by a wide margin. (The 1 s runs in `t7_extra.py` were executed under tracemalloc, which slows Python, so they reached only depth 8 there; the depth reached in the un-instrumented E3 run is the one quoted above.)

E5 Shallow searches (when the original is better)

Depth	Plain (ms)	Redesign (ms)	Redesign ÷ plain
1	0.152	0.231	1.52
2	1.168	0.641	0.55
3	7.071	2.198	0.31
4	49.538	6.035	0.12

E6 Head-to-head (strength)

Redesign (iterative deepening up to a 0.2 s budget per move, mean completed depth 15.9) against plain minimax at depth 4 (the deepest depth that fits 0.2 s), 7×6, 20 games alternating colours, each starting with 4 random plies for variety: **13 wins, 2 draws, 5 losses (score 14.0 / 20)**.

F. Defence of the final design (2 marks)

Objective / prediction	Result	Verdict
O1 / P7 identical root values	0 mismatches in 3,360 tests	Met
O2 / P1 ≥ 10× fewer evaluations at depth 6	70.5× (full); 54.4× without iterative deepening; 11.2× alpha-beta alone	Met; **prediction ranges too low in every component**: alpha-beta alone 11.2× (predicted 3–8), with ordering 34.5× (10–30), full 70.5× (20–60), table alone 4.0× (1.5–3); only the table's marginal gain was inside its range (1.6×, predicted 1.3–2). The transposition ratio at depth 6 (14 %) was lower than predicted, which explains why the table helped more than expected.
P2 ordering worth more than the table	ordering ×3.1 on top of alpha-beta; table ×1.6 on top of ordering	Confirmed
O3 / P3 depth within 1 s ≥ 7 (predicted 7 or 8)	Redesign 10 (hard positions) and up; plain 5 (7×6); plain 7.6 on 5×5	Met, prediction too modest; plain 5×5 depth also higher than predicted (6)
O4 / P4 table < 100 MB (predicted 30–150 MB)	0.1–4.5 MB, 920 entries at depth 7	Met; **prediction wrong by about two orders of magnitude**: good ordering visits only a few thousand nodes, so the table stays small
P5 slower at depth 1–2	1.52× slower at depth 1, **faster** at depth 2 (0.55×)	Half right: the overhead only wins at depth 1
P6 ≥ 14 / 20 against plain depth 4	14.0 / 20 (13 wins, 2 draws, 5 losses)	Met, exactly at the threshold. With 20 games the sampling error is about ±11 percentage points, so this shows an advantage, not its size
P8 exact value only for the best root move	Follows from the root window (children are searched with α = best so far); not separately measured	By design

Magnitude of the improvement. At depth 6 the redesign needs 2,094 nodes and 1,224 evaluations instead of 106,812 and 86,325, i.e. 51× and 70× fewer, and the search time falls from 2.18 s to 0.034 s (64×). The depth that fits the 1 s budget doubles (5 → 10). The gain grows with depth (9.6× at d = 4, 19.4× at d = 5, 70.5× at d = 6) because pruning removes a fraction that grows with the size of the tree, the effect predicted by Task 3 (best-case exponent 0.56 against 1.0).

New costs. (1) Table memory, 125–150 bytes per entry in Python: small here (< 5 MB) but proportional to the visited nodes; for deep searches on a memory-bound device a bounded table with replacement would be needed. (2) Hash maintenance and move sorting on every node; this is why the original is faster at depth 1. (3) About four times as much code and a subtle soundness rule: my first design idea, reusing a deeper stored value, would have changed the root value because win scores contain the remaining depth; the same-depth rule fixes it and was verified by 3,360 tests. (4) A 64-bit key can collide (about 10^{-10} with 10^5 entries); a collision would silently return the value of a different position.

Situations in which the original may still be preferable. (a) **Depth 1** (and trivially small trees): the redesign is 1.5× slower, 0.08 ms in absolute terms, so there is nothing to gain. (b) **When the exact value of every root move is needed** (move ranking, explaining an engine's choice, difficulty levels that deliberately pick the second-best move): alpha-beta returns only bounds for the other moves. (c) **When the evaluation depends on history** (for example a repetition rule) the same position can have different values, which invalidates the transposition table. (d) **When simplicity matters** (teaching code, formal verification) or when memory must be fixed to almost nothing.

Limits of my evidence. The test positions are random mid-game positions, not strategically hard ones (several are decided quickly, which inflates the depths in E3); the head-to-head has only 20 games; the evaluation function itself was not changed, so the redesign makes the search deeper but not wiser; and I

have not optimised the evaluation (82 % of the original cost) by incremental updating, which would be the next step. **Conclusion:** the redesign met all four objectives, the main predictions held in direction but understated the size of the gain, and the one clearly wrong prediction (memory) was wrong in the favourable direction.

Overall Reflection and Summary of Prediction Accuracy

Task	Predictions confirmed	Predictions missed (and the lesson)
1	Open-map counts, maze counts, Greedy on rough terrain, admissibility (0 violations)	Greedy was optimal on the trap map; A* did 34 % of UCS on trap and rough maps (not 40–100 %): the heuristic is sharper than I assumed away from the obstacle.
2	Unreachable-target move count (exact), braiding effect, break-even at M = 2	DFS CR on DFS-carved mazes 1.5 not 3–6 (maze structure); LRTA* first trial 16 not 3–8 and 20–56 trials to converge, not 10–20.
3	Best case, equal values, random range exact	'Worst child first' still pruned (inherited bounds); fixed with the lexicographic tree.
4	Node and leaf formulas, effective branching, growth ×5–7 per ply	Time per node lower than predicted; two depth limits one ply higher; exactness fails at d = 6 on 5-row boards.
5	Failure detected without search (B), pigeonhole factorial growth, check/node trade-off, no solution removed	Instance A left Music with two slots (A2 fixed it); MAC saved 1.6×, not 2–6×; MRV rescue was fragile (14 of 20 seeds).
6	(n−1)! exact, ratio 10 per customer, restarts reach the optimum, boundary n = 10	Time per node 3–10× lower than predicted; single-start local search better than predicted (0–2.3 %).
7	Zero mismatches, ordering worth more than table, tournament threshold	Gains 1.5–3× larger than predicted; table memory two orders of magnitude lower; depth 1 only (not 2) favours the original.

A common thread: **operation counts agreed with the mathematics in every task** (formulas for nodes, leaves, tours, cutoffs were exact where their assumptions held), while **predicted constants and structure-dependent quantities** (time per node, maze structure, tie-breaking, variable-ordering ties) were the usual sources of error. In the Day-4 task, the diagnosis (leaves dominate, transpositions are frequent, no use of bounds) pointed to the redesign, and the redesign was validated against the original on identical positions.

References

[1] S. Russell and P. Norvig, *Artificial Intelligence: A Modern Approach*, 4th ed., Pearson, 2021 (informed search, online search, CSPs, adversarial search, local search).

[2] D. E. Knuth and R. W. Moore, “An analysis of alpha-beta pruning,” *Artificial Intelligence*, vol. 6, no. 4, pp. 293–326, 1975.

[3] R. E. Korf, “Real-time heuristic search,” *Artificial Intelligence*, vol. 42, pp. 189–211, 1990 (LRTA*).

[4] A. K. Mackworth, “Consistency in networks of relations,” *Artificial Intelligence*, vol. 8, pp. 99–118, 1977 (arc consistency).

[5] A. L. Zobrist, “A new hashing method with application for game playing,” Univ. of Wisconsin, Tech. Rep. 88, 1970.

[6] G. A. Croes, “A method for solving traveling-salesman problems,” *Operations Research*, vol. 6, pp. 791–812, 1958 (2-opt).

Appendix A – Source Code of the Implemented Solutions

The experiment drivers (`t1_run.py`, `t2_run.py`, `t2b_run.py`, `t3_run.py`, `t3_predict.py`, `t4_run.py`, `t5_run.py`, `t5_extra.py`, `t6_run.py`, `t7_run.py`, `t7_extra.py`), the figure script, the prediction files and the raw result files (`*.json`, `*_output.txt`) are supplied in the accompanying archive `CSA17_Assignment_2_code_and_data.zip`. Reproduce with `python3 tN_run.py` for N = 1…7.

A.1 Task 1 – grid search (UCS, Greedy, A*) (t1_search.py)

```
"""
CSA17 AI Assignment 2 - Task 1 : Uniform-cost, Greedy best-first and A* on grid maps (campus shuttle)
Author : Saai Vardhan D   Reg. No. 192525205
Grid symbols : '.' road (cost 1)   '~' rough ground (cost 5)   '#' blocked   moves : N,E,S,W (4-connected)
Heuristic    : h(n) = Manhattan distance to goal  (admissible; see report for the relaxed-problem proof)
"""
import heapq, random, time, itertools
MOVES = ((-1, 0), (1, 0), (0, -1), (0, 1))
COST = {'.': 1, '~': 5}
C_MIN = 1


def h(p, g):                        # relaxed problem: no obstacles, every step costs C_MIN
    return C_MIN * (abs(p[0] - g[0]) + abs(p[1] - g[1]))


def search(grid, s, g, algo):
    """algo in {'UCS','GREEDY','ASTAR','ASTAR_FIFO'}.  Returns dict of metrics."""
    R, C = len(grid), len(grid[0]); cnt = itertools.count()
    def key(gc, p):
        hh = h(p, g)
        if algo == 'UCS':        return (gc, next(cnt))
        if algo == 'GREEDY':     return (hh, next(cnt))
        if algo == 'ASTAR':      return (gc + hh, hh, next(cnt))        # ties -> smaller h
        return (gc + hh, next(cnt))                                      # ASTAR_FIFO: ties -> oldest first
    t0 = time.perf_counter()
    best = {s: 0}; parent = {s: None}; heap = [(key(0, s), 0, s)]
    gen, exp, mf = 1, 0, 1; closed = set(); expanded_cells = []
    while heap:
        _, gc, u = heapq.heappop(heap)
        if u in closed: continue
        closed.add(u); exp += 1; expanded_cells.append(u)
        if u == g:
            path = []; v = u
            while v is not None: path.append(v); v = parent[v]
            return dict(found=True, cost=gc, exp=exp, gen=gen, frontier=mf, path=path[::-1], expanded=expanded_cells,
                        time=time.perf_counter() - t0)
        for dr, dc in MOVES:
            v = (u[0] + dr, u[1] + dc)
            if not (0 <= v[0] < R and 0 <= v[1] < C): continue
            ch = grid[v[0]][v[1]]
            if ch == '#': continue
            nc = gc + COST[ch]
            if algo == 'GREEDY':
                if v in best: continue                                   # greedy keeps the first path it finds
            elif v in best and best[v] <= nc: continue
            best[v] = nc; parent[v] = u; gen += 1
            heapq.heappush(heap, (key(nc, v), nc, v))
        mf = max(mf, len(heap))
    return dict(found=False, cost=None, exp=exp, gen=gen, frontier=mf, path=None, expanded=expanded_cells,
time=time.perf_counter() - t0)


# ---------------------------------------------------------------- map generators
def open_map(seed, n=61, p=0.05):
    rng = random.Random(seed)
    while True:
        g = [['.'] * n for _ in range(n)]
        for r in range(n):
            for c in range(n):
                if rng.random() < p: g[r][c] = '#'
        g[1][1] = g[n - 2][n - 2] = '.'
        if search(g, (1, 1), (n - 2, n - 2), 'UCS')['found']: return [''.join(r) for r in g]


def maze_map(seed, n=61):                       # perfect maze by randomised DFS (unique path between any two cells)
    rng = random.Random(seed); g = [['#'] * n for _ in range(n)]
    stack = [(1, 1)]; g[1][1] = '.'
    while stack:
        r, c = stack[-1]
        nb = [(r + dr, c + dc, r + dr // 2, c + dc // 2) for dr, dc in ((-2, 0), (2, 0), (0, -2), (0, 2))
              if 0 < r + dr < n - 1 and 0 < c + dc < n - 1 and g[r + dr][c + dc] == '#']
        if not nb: stack.pop(); continue
        nr, nc, wr, wc = rng.choice(nb); g[wr][wc] = '.'; g[nr][nc] = '.'; stack.append((nr, nc))
    return [''.join(r) for r in g]


def trap_map(n=61):                             # concave cup opening towards the start; goal behind the cup
    g = [['.'] * n for _ in range(n)]
    for c in range(28, 41): g[15][c] = g[45][c] = '#'     # the two arms
    for r in range(15, 46): g[r][40] = '#'                # the back wall of the cup
    return [''.join(r) for r in g]
```

```
def rough_map(n=61):                            # rough band (cost 5) across the middle, cheap road around it (rows 1..3 free)
    g = [['.'] * n for _ in range(n)]
    for r in range(5, n - 1):
        for c in range(20, 41): g[r][c] = '~'
    return [''.join(r) for r in g]

ALGOS = ['UCS', 'GREEDY', 'ASTAR', 'ASTAR_FIFO']
```

A.2 Task 2 – mazes, online DFS, LRTA* (t2_online.py)

```
"""
CSA17 AI Assignment 2 - Task 2 : online search agents (exploration rover)
Author : Saai Vardhan D   Reg. No. 192525205
Maze cells : '#' wall, '.' free.  The rover senses only its 4 neighbours (free or wall) and knows its own coordinates.
"""
import random, collections
DIRS = ((-1, 0), (0, 1), (1, 0), (0, -1))      # N E S W (fixed order -> deterministic agents)


def make_maze(k, seed, braid=0.0):
    """(2k+1)x(2k+1) grid.  Perfect maze by randomised DFS; then a fraction `braid` of dead ends are opened (adds loops)."""
    n = 2 * k + 1; rng = random.Random(seed); g = [['#'] * n for _ in range(n)]
    st = [(1, 1)]; g[1][1] = '.'
    while st:
        r, c = st[-1]
        nb = [(r + dr, c + dc, r + dr // 2, c + dc // 2) for dr, dc in ((-2, 0), (2, 0), (0, -2), (0, 2))
              if 0 < r + dr < n - 1 and 0 < c + dc < n - 1 and g[r + dr][c + dc] == '#']
        if not nb: st.pop(); continue
        nr, nc, wr, wc = rng.choice(nb); g[wr][wc] = '.'; g[nr][nc] = '.'; st.append((nr, nc))
    if braid > 0:
        dead = [(r, c) for r in range(1, n - 1) for c in range(1, n - 1) if g[r][c] == '.' and sum(g[r + dr][c + dc] == '.' for
dr, dc in DIRS) == 1]
        rng.shuffle(dead)
        for (r, c) in dead[:int(round(braid * len(dead)))]:
            if sum(g[r + dr][c + dc] == '.' for dr, dc in DIRS) != 1: continue
            walls = [(r + dr, c + dc, r + 2 * dr, c + 2 * dc) for dr, dc in DIRS
                     if g[r + dr][c + dc] == '#' and 0 < r + 2 * dr < n - 1 and 0 < c + 2 * dc < n - 1 and g[r + 2 * dr][c + 2 *
dc] == '.']
            if walls: wr, wc, _, _ = rng.choice(walls); g[wr][wc] = '.'
    return g


def dead_ends(g):
    n = len(g); return sum(1 for r in range(1, n - 1) for c in range(1, n - 1)
                           if g[r][c] == '.' and sum(g[r + dr][c + dc] == '.' for dr, dc in DIRS) == 1)


def free_cells(g): return [(r, c) for r in range(len(g)) for c in range(len(g)) if g[r][c] == '.']


def bfs_dist(g, s):
    d = {s: 0}; q = collections.deque([s])
    while q:
        u = q.popleft()
        for dr, dc in DIRS:
            v = (u[0] + dr, u[1] + dc)
            if g[v[0]][v[1]] == '.' and v not in d: d[v] = d[u] + 1; q.append(v)
    return d


def neighbours(g, p): return [(p[0] + dr, p[1] + dc) for dr, dc in DIRS if g[p[0] + dr][p[1] + dc] == '.']


# ---------------------------------------------------------------------- online DFS agent
def online_dfs(g, s, target, budget=10**7):
    """Returns (moves, revisits, cells_visited, found). Failure is declared when the agent is back at the start with nothing
unexplored."""
    pos = s; visited = {s}; stack = []; moves = revisits = 0
    while pos != target:
        nxt = next((v for v in neighbours(g, pos) if v not in visited), None)     # untried, unvisited neighbour
        if nxt is not None:
            stack.append(pos); visited.add(nxt); pos = nxt; moves += 1
        elif stack:                                                               # dead end: physically walk back
            pos = stack.pop(); moves += 1; revisits += 1
        else:
            return moves, revisits, len(visited), False                           # explored everything: target unreachable
        if moves > budget: return moves, revisits, len(visited), None
    return moves, revisits, len(visited), True


# ---------------------------------------------------------------------- LRTA* agent (learning real-time A*)
def lrta_trial(g, s, target, H, budget):
    """One trial; H (dict) persists between trials.  Returns (moves, revisits, found)."""
    pos = s; seen = {s}; moves = revisits = 0
    h0 = lambda p: abs(p[0] - target[0]) + abs(p[1] - target[1])
    while pos != target:
        nb = neighbours(g, pos)
        vals = [(1 + H.get(v, h0(v)), i, v) for i, v in enumerate(nb)]
        best = min(vals); H[pos] = best[0]                                        # H(s) <- min_a [c + H(s')]
        pos = best[2]; moves += 1
        if pos in seen: revisits += 1
        seen.add(pos)
        if moves > budget: return moves, revisits, None
    return moves, revisits, True


def pick_targets(g, s):
    d = bfs_dist(g, s); items = sorted(d.items(), key=lambda kv: (kv[1], kv[0]))
    far = items[-1]; near = items[int(0.2 * len(items))]; mid = items[int(0.5 * len(items))]
```

```
    return {'near (20th pct)': near, 'middle (50th pct)': mid, 'far (farthest)': far}, d


def make_unreachable(g, target):
    g2 = [row[:] for row in g]
    for dr, dc in DIRS: g2[target[0] + dr][target[1] + dc] = '#'
    return g2
```

A.3 Task 3 – minimax, alpha-beta, tree builders (t3_alphabeta.py)

```
"""
CSA17 AI Assignment 2 - Task 3 : minimax vs alpha-beta on engineered game trees
Author : Saai Vardhan D   Reg. No. 192525205
Tree = nested lists, leaves are integers (static values).  Root is a MAX node.
"""
import random


class Counter:
    def __init__(self): self.nodes = 0; self.leaves = 0; self.cutoffs = 0


def minimax(t, maxing, c):
    c.nodes += 1
    if not isinstance(t, list): c.leaves += 1; return t
    vals = (minimax(ch, not maxing, c) for ch in t)
    return max(vals) if maxing else min(vals)


def alphabeta(t, maxing, alpha, beta, c):
    c.nodes += 1
    if not isinstance(t, list): c.leaves += 1; return t
    if maxing:
        v = float('-inf')
        for ch in t:
            v = max(v, alphabeta(ch, False, alpha, beta, c)); alpha = max(alpha, v)
            if alpha >= beta: c.cutoffs += 1; break                # beta cutoff
        return v
    v = float('inf')
    for ch in t:
        v = min(v, alphabeta(ch, True, alpha, beta, c)); beta = min(beta, v)
        if alpha >= beta: c.cutoffs += 1; break                    # alpha cutoff
    return v


def build(b, d, leaves):                                           # leaves: iterator of leaf values
    if d == 0: return next(leaves)
    return [build(b, d - 1, leaves) for _ in range(b)]


def value(t, maxing):                                              # true minimax value (used only to ORDER the children)
    if not isinstance(t, list): return t
    vs = [value(ch, not maxing) for ch in t]; return max(vs) if maxing else min(vs)


def order(t, maxing, best_first):
    """Re-order children of every node by true minimax value. best_first=True: best move for the player to move first (perfect
ordering)."""
    if not isinstance(t, list): return t
    kids = [order(ch, not maxing, best_first) for ch in t]
    kids.sort(key=lambda k: value(k, not maxing), reverse=(maxing == best_first))
    return kids


def lexi_tree(b, d):
    """Adversarial tree: leaf value = sum_j s_j * i_j * B^(d-j)  with s_j = +1 for MAX levels, -1 for MIN levels (i_j = index of
the child taken at level j).
    Every later child is strictly better for the player to move AND every leaf lies inside the (alpha, beta) window of ALL its
ancestors,
    so no cutoff can ever happen (the leading term at each level dominates all lower levels)."""
    B = b + 1
    def rec(level, acc):
        if level == d: return acc
        sign = 1 if level % 2 == 0 else -1
        return [rec(level + 1, acc + sign * i * B ** (d - level - 1)) for i in range(b)]
    return rec(0, 0)


def make_tree(b, d, kind, seed=0):
    rng = random.Random(seed); n = b ** d
    if kind == 'lexi': return lexi_tree(b, d)
    if kind == 'equal': return build(b, d, iter([5] * n))
    perm = list(range(1, n + 1)); rng.shuffle(perm)                 # distinct leaf values -> distinct node values
    t = build(b, d, iter(perm))
    if kind == 'random': return t
    if kind == 'best':   return order(t, True, True)
    if kind == 'worst':  return order(t, True, False)
    raise ValueError(kind)


def run(t):
    cm = Counter(); vm = minimax(t, True, cm)
    ca = Counter(); va = alphabeta(t, True, float('-inf'), float('inf'), ca)
    assert vm == va, "alpha-beta must return the minimax value"
    return cm, ca, vm
```

A.4 Task 4 – Connect-K game and minimax (t4_game.py)

```
"""
CSA17 AI Assignment 2 - Task 4 : depth-limited minimax for Connect-K (gravity)
Author : Saai Vardhan D   Reg. No. 192525205
Board : W columns x H rows, players 1 (maximiser = 'me') and 2.  A move drops a piece into a column.
"""
import random, time


WIN = 1_000_000


class Game:
    def __init__(self, W, H, K):
        self.W, self.H, self.K = W, H, K
        self.g = [[0] * W for _ in range(H)]          # g[row][col], row 0 = bottom
        self.h = [0] * W                              # column heights
        self.windows = []                             # all K-in-a-row windows as lists of (r,c)
        for r in range(H):
            for c in range(W):
                for dr, dc in ((0, 1), (1, 0), (1, 1), (1, -1)):
                    er, ec = r + dr * (K - 1), c + dc * (K - 1)
                    if 0 <= er < H and 0 <= ec < W: self.windows.append([(r + dr * i, c + dc * i) for i in range(K)])
        self.centre = W // 2
        self.moves_played = 0


    def legal(self): return [c for c in range(self.W) if self.h[c] < self.H]
    def play(self, c, p): r = self.h[c]; self.g[r][c] = p; self.h[c] += 1; self.moves_played += 1; return r
    def undo(self, c): self.h[c] -= 1; self.g[self.h[c]][c] = 0; self.moves_played -= 1
    def full(self): return self.moves_played == self.W * self.H


    def wins(self, r, c, p):                          # does the piece just placed at (r,c) complete K in a row?
        g, K = self.g, self.K
        for dr, dc in ((0, 1), (1, 0), (1, 1), (1, -1)):
            n = 1
            for s in (1, -1):
                rr, cc = r + s * dr, c + s * dc
                while 0 <= rr < self.H and 0 <= cc < self.W and g[rr][cc] == p: n += 1; rr += s * dr; cc += s * dc
            if n >= K: return True
        return False


    # ---------------- evaluation function (my own design) ----------------
    SCORE = (0, 1, 10, 100, 100000)
    def evaluate(self, me):
        """Window heuristic: a window containing only my pieces scores SCORE[count]; only opponent pieces scores -SCORE[count];
        mixed or empty windows score 0.  Plus 3 points for every piece of mine in the centre column minus 3 for the
opponent's."""
        opp = 3 - me; g = self.g; s = 0; S = self.SCORE
        for w in self.windows:
            a = b = 0
            for (r, c) in w:
                v = g[r][c]
                if v == me: a += 1
                elif v == opp: b += 1
            if b == 0: s += S[a]
            elif a == 0: s -= S[b]
        cc = self.centre
        for r in range(self.h[cc]):
            v = g[r][cc]
            if v == me: s += 3
            elif v == opp: s -= 3
        return s


class Count:
    def __init__(self): self.nodes = 0; self.evals = 0; self.terminals = 0


def minimax(game, depth, player, me, cnt, last=None):
    """Plain depth-limited minimax. Returns the value from the point of view of `me`.  `last` = (col,row,player) of the move
that led here."""
    cnt.nodes += 1
    if last is not None and game.wins(last[1], last[0], last[2]):
        cnt.terminals += 1; return (WIN + depth) if last[2] == me else -(WIN + depth)     # earlier win scores higher
    moves = game.legal()
    if not moves: cnt.terminals += 1; return 0                                           # draw
    if depth == 0: cnt.evals += 1; return game.evaluate(me)                              # cut-off: use the evaluation function
    best = -10 ** 9 if player == me else 10 ** 9
    for c in moves:
        r = game.play(c, player)
        v = minimax(game, depth - 1, 3 - player, me, cnt, (c, r, player))
        game.undo(c)
        if player == me: best = max(best, v)
        else: best = min(best, v)
    return best


def best_move(game, depth, me, cnt):
    best, bm = -10 ** 9, None
    for c in game.legal():
        r = game.play(c, me)
        v = minimax(game, depth - 1, 3 - me, me, cnt, (c, r, me)); game.undo(c)
        if v > best: best, bm = v, c
    return bm, best


def random_position(game_args, plies, seed):
    rng = random.Random(seed)
```

```
    while True:
        g = Game(*game_args); p = 1; ok = True
        for _ in range(plies):
            mv = rng.choice(g.legal()); r = g.play(mv, p)
            if g.wins(r, mv, p) or g.full(): ok = False; break
            p = 3 - p
        if ok: return g, p


def play_vs_random(game_args, depth, seed, minimax_first):
    rng = random.Random(seed); g = Game(*game_args); p = 1; me = 1 if minimax_first else 2
    while True:
        if p == me: c, _ = best_move(g, depth, me, Count())
        else: c = rng.choice(g.legal())
        r = g.play(c, p)
        if g.wins(r, c, p): return 1 if p == me else -1
        if g.full(): return 0
        p = 3 - p
```

A.5 Task 5 – CSP, propagation, BT, MAC (t5_csp.py)

```
"""
CSA17 AI Assignment 2 - Task 5 : exam timetabling CSP with constraint propagation derived from the support principle
Author : Saai Vardhan D   Reg. No. 192525205
Representation
  n variables 0..n-1 (exams), domains[i] = set of slots.  Binary constraints stored per ARC:  rel[(i,j)] = f(a,b) -> bool,  a is
a slot of exam i, b a slot of exam j.
  nbrs[i] = list of j that share a constraint with i.  conflict (shared students): a != b.  precedence i before j: a < b  (and
the reverse arc b > a).
"""
import collections, random


NE = lambda a, b: a != b
LT = lambda a, b: a < b
GT = lambda a, b: a > b


class CSP:
    def __init__(self, n, slots, names=None):
        self.n = n; self.slots = slots; self.domains = [set(range(slots)) for _ in range(n)]
        self.rel = {}; self.nbrs = [[] for _ in range(n)]; self.names = names or [f"E{i}" for i in range(n)]
    def add(self, i, j, f_ij, f_ji):
        self.rel[(i, j)] = f_ij; self.rel[(j, i)] = f_ji
        if j not in self.nbrs[i]: self.nbrs[i].append(j)
        if i not in self.nbrs[j]: self.nbrs[j].append(i)
    def conflict(self, i, j): self.add(i, j, NE, NE)
    def before(self, i, j): self.add(i, j, LT, GT)                    # slot(i) < slot(j)


class Stats:
    def __init__(self): self.checks = 0; self.revisions = 0; self.removals = 0; self.wipeouts = 0; self.nodes = 0;
self.backtracks = 0; self.history = None


def revise(csp, dom, i, j, S):
    """Remove from D(i) every value that has NO supporting value in D(j).  Returns True if D(i) changed."""
    f = csp.rel[(i, j)]; removed = []
    for a in dom[i]:
        supported = False
        for b in dom[j]:
            S.checks += 1
            if f(a, b): supported = True; break
        if not supported: removed.append(a)
    for a in removed: dom[i].discard(a)
    S.removals += len(removed); S.revisions += 1
    return bool(removed)


def propagate(csp, dom, queue, S):
    """Arc-consistency propagation (my derivation of the principle).  queue: iterable of arcs (i,j) to (re)examine.
    Returns False on a domain wipe-out (no solution), True otherwise."""
    q = collections.deque(queue); inq = set(q)
    while q:
        i, j = q.popleft(); inq.discard((i, j))
        if revise(csp, dom, i, j, S):
            if not dom[i]: S.wipeouts += 1; return False                # empty domain -> no solution below this point
            if S.history is not None: S.history.append((len(S.history), [set(d) for d in dom], f"revise D({csp.names[i]})
against {csp.names[j]}"))
            for k in csp.nbrs[i]:
                if k != j and (k, i) not in inq: q.append((k, i)); inq.add((k, i))      # supports of D(k) may have depended on
the removed values
    return True


def all_arcs(csp): return list(csp.rel.keys())


# ---------------------------------------------------------------------- search
def consistent(csp, assign, i, a, S):
    for j in csp.nbrs[i]:
        if j in assign:
            S.checks += 1
            if not csp.rel[(i, j)](a, assign[j]): return False
    return True


def bt(csp, S, order=None, cap=100000):
    """Plain chronological backtracking, static variable order, values in increasing order, NO propagation (domains used as
given)."""
```

```
    order = order or list(range(csp.n)); assign = {}
    def rec(k):
        if k == len(order): return True
        i = order[k]; ok_any = False
        for a in sorted(csp.domains[i]):
            if S.nodes >= cap: return None
            if consistent(csp, assign, i, a, S):
                ok_any = True; S.nodes += 1; assign[i] = a
                r = rec(k + 1)
                if r: return True
                if r is None: return None
                del assign[i]; S.backtracks += 1
        if not ok_any: S.wipeouts += 1                                  # dead end: every value of this exam conflicts with
earlier choices
        return False
    r = rec(0); return (dict(assign) if r else None), r


def mac(csp, S, mrv=False, cap=100000):
    """Maintaining arc consistency: propagate once at the root, then after every assignment re-examine only the arcs (k,i)
pointing at the assigned exam."""
    dom = [set(d) for d in csp.domains]
    if not propagate(csp, dom, all_arcs(csp), S): return None, False
    def pick(dom, assigned):
        free = [i for i in range(csp.n) if i not in assigned]
        if not mrv: return free[0]
        return min(free, key=lambda i: (len(dom[i]), -len(csp.nbrs[i]), i))        # fewest remaining values, then most
constraints
    def rec(dom, assigned):
        if len(assigned) == csp.n: return True
        i = pick(dom, assigned)
        for a in sorted(dom[i]):
            if S.nodes >= cap: return None
            S.nodes += 1; nd = [set(d) for d in dom]; nd[i] = {a}; assigned[i] = a
            if propagate(csp, nd, [(k, i) for k in csp.nbrs[i]], S):
                r = rec(nd, assigned)
                if r: return True
                if r is None: return None
            del assigned[i]; S.backtracks += 1
        return False
    assigned = {}; r = rec(dom, assigned)
    return (dict(assigned) if r else None), r


def verify(csp, sol):
    return all(csp.rel[(i, j)](sol[i], sol[j]) for (i, j) in csp.rel if i < j) and all(sol[i] in csp.domains[i] for i in
range(csp.n))


def solutions(csp, limit=10**6):
    sols = []; assign = {}
    def rec(i):
        if i == csp.n: sols.append(dict(assign)); return
        for a in csp.domains[i]:
            if all(csp.rel[(i, j)](a, assign[j]) for j in csp.nbrs[i] if j in assign): assign[i] = a; rec(i + 1); del assign[i]
    rec(0); return sols


# ---------------------------------------------------------------------- instances
def instance_A():
    """Propagation alone solves it.  8 exams, 4 slots (0..3). Maths is fixed in slot 0; chain Maths < Physics < Chemistry <
Biology; conflicts for the rest."""
    c = CSP(8, 4, ["Maths", "Physics", "Chem", "Biology", "English", "History", "Art", "Music"])
    c.domains[0] = {0}                                          # unary: Maths only in slot 0 (hall booked)
    c.before(0, 1); c.before(1, 2); c.before(2, 3)              # syllabus order
    for j in (0, 1, 2): c.conflict(4, j)                         # English shares students with Maths, Physics, Chem
    c.conflict(5, 3); c.conflict(5, 4); c.conflict(5, 1); c.conflict(5, 2)
    c.conflict(6, 5); c.conflict(6, 1); c.conflict(6, 2)
    c.conflict(7, 0); c.conflict(7, 5); c.conflict(7, 6)
    return c


def instance_B():
    """Propagation alone detects that there is no timetable."""
    c = CSP(4, 3, ["A", "B", "C", "D"]); c.domains[0] = {0}; c.domains[1] = {0, 1}; c.domains[2] = {0, 1}
    for i, j in ((0, 1), (0, 2), (1, 2), (3, 0), (3, 1), (3, 2)): c.conflict(i, j)
    return c


def pigeonhole(k):
    c = CSP(k + 1, k)
    for i in range(k + 1):
        for j in range(i + 1, k + 1): c.conflict(i, j)
    return c


def hidden_core(free=12, slots=4, seed=5):
    """`free` loosely connected exams first in the order, then a clique of slots+1 exams (unsatisfiable with `slots` slots)
last."""
    rng = random.Random(seed); n = free + slots + 1; c = CSP(n, slots)
    for i in range(free):
        for j in range(i + 1, free):
            if rng.random() < 0.12: c.conflict(i, j)
    for i in range(free, n):
        for j in range(i + 1, n): c.conflict(i, j)
    return c


def random_timetable(n, slots, avg_deg, seed):
    rng = random.Random(seed); c = CSP(n, slots); p = avg_deg / (n - 1)
    for i in range(n):
        for j in range(i + 1, n):
            if rng.random() < p: c.conflict(i, j)
```

```
    return c
```

A.6 Task 6 – exhaustive TSP and 2-opt (t6_tsp.py)

```
"""
CSA17 AI Assignment 2 - Task 6 : delivery tour planning, exhaustive search vs 2-opt local search
Author : Saai Vardhan D   Reg. No. 192525205
Representation: tour = list [0, c1, ..., c(n-1), 0] ; depot = node 0.  dist = full Euclidean matrix.
"""
import math, random, time, itertools


def make_instance(n, seed):
    rng = random.Random(seed); pts = [(rng.uniform(0, 100), rng.uniform(0, 100)) for _ in range(n)]
    return pts, [[math.dist(a, b) for b in pts] for a in pts]


def tour_length(d, tour): return sum(d[tour[i]][tour[i + 1]] for i in range(len(tour) - 1))


# ---------------------------------------------------------------------- exhaustive enumeration (my own design)
def exhaustive(d):
    """Generate every permutation of customers 1..n-1 by recursive extension; return (best_cost, best_tour, tours_enumerated,
tree_nodes)."""
    n = len(d); best = [float('inf'), None]; stat = [0, 0]               # [tours, nodes]
    path = [0]; used = [False] * n; used[0] = True
    def rec(last, cost, depth):
        stat[1] += 1
        if depth == n - 1:                                               # complete permutation: close the tour at the depot
            tot = cost + d[last][0]; stat[0] += 1
            if tot < best[0]: best[0] = tot; best[1] = path + [0]
            return
        row = d[last]
        for c in range(1, n):
            if not used[c]:
                used[c] = True; path.append(c)
                rec(c, cost + row[c], depth + 1)
                path.pop(); used[c] = False
    rec(0, 0.0, 0)
    return best[0], best[1], stat[0], stat[1]


# ---------------------------------------------------------------------- independent checkers
def brute_force_itertools(d):
    n = len(d); best = float('inf')
    for p in itertools.permutations(range(1, n)):
        c = d[0][p[0]] + sum(d[p[i]][p[i + 1]] for i in range(n - 2)) + d[p[-1]][0]
        best = min(best, c)
    return best


def held_karp(d):                                                        # dynamic programming optimum, used ONLY as an
independent verifier
    n = len(d); INF = float('inf'); dp = {}
    for j in range(1, n): dp[(1 << j, j)] = d[0][j]
    for size in range(2, n):
        for sub in itertools.combinations(range(1, n), size):
            m = sum(1 << j for j in sub)
            for j in sub:
                pm = m ^ (1 << j); dp[(m, j)] = min(dp[(pm, k)] + d[k][j] for k in sub if k != j)
    full = sum(1 << j for j in range(1, n))
    return min(dp[(full, j)] + d[j][0] for j in range(1, n))


# ---------------------------------------------------------------------- local search : 2-opt hill climbing with random restarts
def two_opt(d, tour):
    """First-improvement 2-opt: reverse the segment tour[i..j] whenever it shortens the tour. Returns (tour, n_moves,
n_evals)."""
    n = len(tour); improved = True; moves = evals = 0
    while improved:
        improved = False
        for i in range(1, n - 2):
            for j in range(i + 1, n - 1):
                evals += 1
                a, b, c, e = tour[i - 1], tour[i], tour[j], tour[j + 1]
                if d[a][c] + d[b][e] < d[a][b] + d[c][e] - 1e-12:
                    tour[i:j + 1] = reversed(tour[i:j + 1]); improved = True; moves += 1
    return tour, moves, evals


def hill_climb(d, rng, restarts=1):
    n = len(d); best = (float('inf'), None); total_evals = 0
    for _ in range(restarts):
        cust = list(range(1, n)); rng.shuffle(cust); t = [0] + cust + [0]
        t, mv, ev = two_opt(d, t); total_evals += ev; L = tour_length(d, t)
        if L < best[0]: best = (L, t[:])
    return best[0], best[1], total_evals


def nearest_neighbour(d):
    n = len(d); t = [0]; left = set(range(1, n))
    while left: nx = min(left, key=lambda c: d[t[-1]][c]); t.append(nx); left.remove(nx)
    return t + [0]
```

A.7 Task 7 – redesigned search (t7_improved.py)

```
"""
CSA17 AI Assignment 2 - Task 7 : redesigned search for the Connect-K agent of Task 4
Author : Saai Vardhan D   Reg. No. 192525205
Components : alpha-beta (C1), centre-first ordering (C2), transposition table with Zobrist keys and EXACT/LOWER/UPPER flags
(C3),
             iterative deepening with previous best move first (C4).   Each can be switched off for the ablation study.
"""
import random, time
from t4_game import Game, WIN, Count, minimax, best_move


INF = 10 ** 9
EXACT, LOWER, UPPER = 0, 1, 2


class ZGame(Game):
    """Game with an incrementally maintained 64-bit Zobrist key (xor of one random number per (cell, player))."""
    def __init__(self, W, H, K, seed=2024):
        super().__init__(W, H, K)
        rng = random.Random(seed)
        self.Z = [[[rng.getrandbits(64) for _ in range(3)] for _ in range(W)] for _ in range(H)]
        self.key = 0
    def play(self, c, p):
        r = super().play(c, p); self.key ^= self.Z[r][c][p]; return r
    def undo(self, c):
        r = self.h[c] - 1; p = self.g[r][c]; self.key ^= self.Z[r][c][p]; super().undo(c)


def clone(game):
    z = ZGame(game.W, game.H, game.K)
    for r in range(game.H):
        for c in range(game.W):
            v = game.g[r][c]
            if v:
                z.g[r][c] = v; z.key ^= z.Z[r][c][v]
    z.h = game.h[:]; z.moves_played = game.moves_played
    return z


class Cfg:
    def __init__(self, name, ab=True, centre=True, tt=True, iterative=True):
        self.name, self.ab, self.centre, self.tt, self.iterative = name, ab, centre, tt, iterative


CONFIGS = {
    'plain (Task 4)':          Cfg('plain (Task 4)', ab=False, centre=False, tt=False, iterative=False),
    'TT only':                 Cfg('TT only', ab=False, centre=False, tt=True, iterative=False),
    'alpha-beta':              Cfg('alpha-beta', ab=True, centre=False, tt=False, iterative=False),
    'alpha-beta + centre':     Cfg('alpha-beta + centre', ab=True, centre=True, tt=False, iterative=False),
    'alpha-beta + centre + TT': Cfg('alpha-beta + centre + TT', ab=True, centre=True, tt=True, iterative=False),
    'FULL (+ iterative deepening)': Cfg('FULL (+ iterative deepening)', ab=True, centre=True, tt=True, iterative=True),
}


class Stats:
    def __init__(self): self.nodes = 0; self.evals = 0; self.terminals = 0; self.tt_hits = 0; self.deadline = None


class Timeout(Exception): pass


def search(g, depth, player, me, alpha, beta, st, cfg, tt, last):
    st.nodes += 1
    if st.deadline is not None and (st.nodes & 255) == 0 and time.perf_counter() > st.deadline: raise Timeout
    if last is not None and g.wins(last[1], last[0], last[2]):
        st.terminals += 1; return (WIN + depth) if last[2] == me else -(WIN + depth)
    moves = g.legal()
    if not moves: st.terminals += 1; return 0
    if depth == 0: st.evals += 1; return g.evaluate(me)
    a0, b0 = alpha, beta; ttmove = None
    if cfg.tt:
        e = tt.get(g.key)
        if e is not None:
            ed, flag, val, mv = e; ttmove = mv
            if ed == depth:                                  # SAME remaining depth -> value is usable
                if flag == EXACT: st.tt_hits += 1; return val
                if flag == LOWER and val > alpha: alpha = val
                elif flag == UPPER and val < beta: beta = val
                if alpha >= beta: st.tt_hits += 1; return val
    if cfg.centre:
        c0 = g.centre; moves = sorted(moves, key=lambda c: abs(c - c0))
    if ttmove is not None and ttmove in moves:
        moves = [ttmove] + [c for c in moves if c != ttmove]
    maxing = (player == me); best = -INF if maxing else INF; bm = None
    for c in moves:
        r = g.play(c, player)
        v = search(g, depth - 1, 3 - player, me, alpha, beta, st, cfg, tt, (c, r, player))
        g.undo(c)
        if maxing:
            if v > best: best, bm = v, c
            if cfg.ab and best > alpha: alpha = best
        else:
            if v < best: best, bm = v, c
            if cfg.ab and best < beta: beta = best
        if cfg.ab and alpha >= beta: break
    if cfg.tt:
```

```
        flag = UPPER if best <= a0 else (LOWER if best >= b0 else EXACT)
        if not cfg.ab: flag = EXACT
        tt[g.key] = (depth, flag, best, bm)
    return best


def root(g, depth, me, st, cfg, tt):
    """Returns (best move, best value). Only the best move's value is exact when alpha-beta is on."""
    moves = g.legal()
    if cfg.centre: moves = sorted(moves, key=lambda c: abs(c - g.centre))
    if cfg.tt:
        e = tt.get(g.key)
        if e is not None and e[3] in moves: moves = [e[3]] + [c for c in moves if c != e[3]]
    elif cfg.iterative and getattr(tt, 'pv', None) in moves:
        moves = [tt.pv] + [c for c in moves if c != tt.pv]
    best, bm = -INF, None
    for c in moves:
        r = g.play(c, me)
        v = search(g, depth - 1, 3 - me, me, best if cfg.ab else -INF, INF, st, cfg, tt, (c, r, me)); g.undo(c)
        if v > best or bm is None: best, bm = v, c
    if cfg.tt: tt[g.key] = (depth, EXACT, best, bm)
    return bm, best


def decide(game, depth, me, cfg, st=None, tt=None, time_limit=None):
    """Fixed-depth decision (iterative configs run depths 1..depth and all work is counted) or, with time_limit,
       iterative deepening until the budget is used up.  Returns (move, value, completed_depth, stats, tt)."""
    g = clone(game) if not isinstance(game, ZGame) else game
    st = st or Stats(); tt = tt if tt is not None else {}
    if time_limit is not None: st.deadline = time.perf_counter() + time_limit
    depths = range(1, depth + 1) if cfg.iterative else [depth]
    mv = val = None; done = 0
    try:
        for d in depths:
            mv, val = root(g, d, me, st, cfg, tt); done = d
    except Timeout:
        pass                                            # board was restored? undo is skipped by the exception -> caller used
a clone
    return mv, val, done, st, tt


def plain_root_values(game, depth, me):
    """Exact minimax value of every root move (for the tie-aware correctness check)."""
    vals = {}
    for c in game.legal():
        r = game.play(c, me); vals[c] = minimax(game, depth - 1, 3 - me, me, Count(), (c, r, me)); game.undo(c)
    return vals
```

Appendix B – Predictions and Complete Console Output of Each Task

Each output file starts with the prediction text that was written before the experiment was run, followed by the observed results.

B.1 Task 1 (t1_output.txt)

```
TASK 1 - PREDICTIONS (written BEFORE any experiment was run)
Maps: 61x61 grids, 4-connected moves, road cell cost 1, rough cell cost 5, '#' blocked. Start top-left, goal bottom-right (maps 1,2), see
map for 3,4.
Heuristic h(n) = Manhattan distance (relaxed problem: no obstacles, every cell costs the minimum 1).


MAP 1  OPEN (5% random obstacles), start (1,1) goal (59,59), optimal cost about 116-120
  UCS    : explores a disc/diamond of every cell with g <= C*, i.e. almost all free cells (predict 3,000-3,500 expansions). Optimal.
  Greedy : follows h straight to the goal, about as many expansions as the path length (predict 120-200). Cost near optimal (0-10% above).
  A*     : with ties broken toward smaller h it expands a narrow band (predict 120-400). With FIFO tie-breaking every cell in the
           rectangle has f = C*, so it degrades towards UCS (predict 1,500-3,300). Optimal both ways.
MAP 2  MAZE (perfect maze, unique path), start (1,1) goal (59,59)
  The straight-line heuristic gives little guidance. UCS expands every cell closer than the goal along the maze path (predict 50-100% of
cells).
  A* about the same as UCS (predict 60-100% of UCS). Greedy gets pulled into dead ends toward the goal side (predict 40-90% of cells).
  All three return the same cost because the path is unique (all optimal).
MAP 3  CONCAVE TRAP (cup-shaped wall opening towards the start, goal behind the cup)
  Greedy rushes into the cup, fills it (dead end), then escapes: predict expansions >> path length (300-900) and a path that is NOT optimal
  or far longer than needed. A* expands everything with f < C*: predict 40-70% of UCS. UCS biggest. A* and UCS optimal.
MAP 4  ROUGH TERRAIN (middle band of cost-5 cells; cheap road detour exists around the band)
  Greedy walks straight through the rough band: predict cost >= 40% above optimal. A* is optimal but h (=1 per cell) badly
  under-estimates rough cells, so predict A* expansions close to UCS (70-100% of UCS). UCS optimal, many expansions.
DESIGN DECISION: with 95% open-road trips I predict A* (tie-break on h) is the best choice: greedy-like cost on open maps but guaranteed
optimal.


================================================================================================
OBSERVED
================================================================================================


MAP 1  OPEN ROAD (5% random obstacles) - mean over 30 random maps (seeds 0-29), start (1,1) goal (59,59)
  algo         expanded      (min-max)  generated max frontier mean cost  optimal in  time ms
  UCS              3533   (3500, 3556)       3535           62     116.0       30/30      6.8
  GREEDY            123     (117, 135)        332          210     120.3        2/30      0.6
  ASTAR             178     (117, 404)        433          256     116.0       30/30      0.5
  ASTAR_FIFO       3104   (2950, 3296)       3318          215     116.0       30/30      6.4


MAP 2  PERFECT MAZE - mean over 30 mazes (seeds 0-29), start (1,1) goal (59,59); free cells in maze 0 = 1799
  algo         expanded      (min-max)  generated max frontier mean cost  optimal in  time ms
  UCS              1046    (331, 1799)       1048            6     632.7       30/30      1.7
  GREEDY            778    (295, 1531)        790           13     632.7       30/30      1.3
  ASTAR             988    (298, 1786)        991            6     632.7       30/30      1.7
  ASTAR_FIFO        992    (299, 1789)        995            6     632.7       30/30      1.6


MAP 3  CONCAVE TRAP (cup opens towards start)   start=(30, 5) goal=(30, 55)  optimal cost (UCS) = 82
  algo         expanded  generated max frontier   cost  optimal?  time ms
  UCS              3615       3632           72     82      True      6.6
  GREEDY            686        789          104     82      True      1.5
  ASTAR            1231       1343          113     82      True      2.5
  ASTAR_FIFO       1763       1907          145     82      True      3.2


MAP 4  ROUGH TERRAIN (cost-5 band, columns 20-40, rows 5-59; road above)   start=(40, 5) goal=(40, 55)  optimal cost (UCS) = 90
  algo         expanded  generated max frontier   cost  optimal?  time ms
  UCS              2959       3093          135     90      True      5.5
  GREEDY             51        152          102    134     False      0.1
  ASTAR             998       1162          165     90      True      2.1
  ASTAR_FIFO       1316       1454          139     90      True      2.6


HEURISTIC CHECK: h(n) <= true cost-to-goal for every cell reachable on maps 3 and 4 (reverse Dijkstra from the goal)
  trap  : cells checked= 3666  violations of h<=h*: 0
  rough : cells checked= 3721  violations of h<=h*: 0
  open0 : cells checked= 3504  violations of h<=h*: 0
  maze0 : cells checked= 1799  violations of h<=h*: 0


WORKLOAD MODEL: 95% open-road trips + 5% 'other' trips (mean of maze, trap, rough maps)
  algo         E[expansions]  E[excess cost %]
  UCS                   3483             0.00
  GREEDY                 142             4.36
  ASTAR                  223             0.00
  ASTAR_FIFO            3017             0.00


saved.
```

B.2 Task 2 (experiments 1–6) (t2_output.txt)

```
TASK 2 - PREDICTIONS (written BEFORE any experiment was run)
Setting: rover in an unknown maze, senses only which of its 4 neighbouring cells are free, knows its own coordinates and the dock
coordinates.
Agents: online DFS (explore, backtrack when stuck) and LRTA* (h0 = Manhattan distance, learned values persist across trials).
Competitive ratio CR = moves / optimal path length with full map.


P1 Perfect 21x21 maze (221 free cells): online DFS far-target CR about 3-6 (it explores most of the maze: roughly 2 moves per explored
cell);
   near target (20th percentile distance) CR higher (5-10) with large variance because the DFS order is blind to where the dock is.
```

```
P2 LRTA* first trial: similar order of magnitude as DFS (CR 3-8). Over repeated trials LRTA* moves fall and CR -> 1.0 within about 10-20
trials.
   Online DFS gives no learning: identical moves every trial.
P3 Unreachable target in a perfect maze: online DFS terminates after exactly 2*(cells-1) moves (each corridor walked twice) = 440 on 21x21.
   LRTA* never terminates (it keeps raising heuristic values): predicted to exhaust the move budget.
P4 Maze size: CR of online DFS grows with the maze size (cells grow like size^2 while the optimal path grows slower) - predict CR roughly
doubles from 11x11 to 41x41.
P5 Dead-end density: braiding (removing dead ends = adding loops) shortens the optimal path, but DFS still explores much of the maze,
   so predict CR of online DFS INCREASES when dead ends are removed.
P6 Changed condition (20 facilities, static maps for months): exploring once and then planning on the stored map costs
   moves_DFS + (M-1)*optimal for M missions. I predict break-even against repeated online DFS already at M = 2 missions, and against
   LRTA* (which needs about 10+ trials to converge) at M of about 3-10.


================================================================================================
OBSERVED
================================================================================================


E1  TARGET DISTANCE  21x21 perfect mazes, 30 mazes (seeds 0-29)
  target             opt len | DFS moves  revisits     CR | LRTA* t1 moves     CR | LRTA* t10 moves     CR
  near (20th pct)       35.3 |     108.8      36.7   3.12 |          107.8   3.10 |            41.9   1.18
  middle (50th pct)     76.0 |     114.9      19.5   1.53 |          562.5   7.45 |           112.5   1.47
  far (farthest)       136.5 |     198.4      30.9   1.46 |         2185.7  15.81 |           285.6   2.11


E2  MAZE SIZE  perfect mazes, far target, 30 mazes per size
     size  cells  opt len | DFS moves     CR | LRTA* t1 moves     CR | LRTA* t10 CR
   11x11      49     38.0 |      47.5   1.26 |          179.7   4.77 |         1.24
   21x21     199    136.5 |     198.4   1.46 |         2185.7  15.81 |         2.11
   31x31     449    274.3 |     431.4   1.59 |         6562.9  24.03 |         2.49
   41x41     799    463.9 |     845.5   1.84 |        15921.9  34.70 |         2.94


E3  DEAD-END DENSITY  31x31 mazes (k=15), far target, braid = fraction of dead ends opened into loops, 30 mazes each
  braid  dead ends  opt len | DFS moves     CR | LRTA* t1 moves     CR
   0.00       24.3    274.3 |     431.4   1.59 |         6562.9  24.03
   0.25       18.1    168.5 |     386.5   2.48 |         4499.0  25.66
   0.50       12.0    128.9 |     339.6   2.92 |         3195.1  24.39
   0.75        5.9    104.9 |     306.6   3.14 |         1918.9  17.59
   1.00        0.0     88.7 |     244.3   2.88 |         1553.6  15.35


E4  UNREACHABLE TARGET  perfect mazes, target cell sealed in by walls; LRTA* budget = 60 x free cells per trial
     size reachable cells  DFS moves  2*(cells-1)  DFS declares failure |                                LRTA* result
   11x11             47.0       92.0         92.0            30/30 correct | LRTA* exhausted budget in 30/30 mazes (never concluded)
   21x21            197.0      392.0        392.0            30/30 correct | LRTA* exhausted budget in 30/30 mazes (never concluded)
   31x31            447.0      892.0        892.0            30/30 correct | LRTA* exhausted budget in 30/30 mazes (never concluded)


E5  LEARNING ACROSS TRIALS  21x21 perfect mazes, far target, mean over 30 mazes
  optimal length (mean) = 136.5   online DFS moves per trial (same every trial) = 198.4
  trial :     1     2     3     4     5     6     8    10    15    20    25
  LRTA* : 2185.7  288.9  292.1  272.4  287.4  301.0  269.3  285.6  271.3  243.5  240.9
  CR    :  16.01   2.12   2.14   2.00   2.10   2.20   1.97   2.09   1.99   1.78   1.76


E6  CHANGED CONDITION: 20 rovers / 20 facilities, static maps.  Total moves for ONE facility over M missions (mean over 20 mazes, far
target)


  facility 21x21: optimal=133.7  online-DFS per mission=195.9  LRTA* trial1=1969.1
  M missions |  DFS every mission  LRTA* (learns)  DFS once + stored map |  moves saved vs DFS
           1 |               196            1969                   196 |               0.0%
           2 |               392            2257                   330 |              15.9%
           3 |               588            2551                   463 |              21.2%
           5 |               980            3097                   731 |              25.4%
          10 |              1959            4468                  1399 |              28.6%
          20 |              3918            7139                  2736 |              30.2%
          50 |              9795           13178                  6747 |              31.1%


  facility 41x41: optimal=466.9  online-DFS per mission=850.9  LRTA* trial1=17926.4
  M missions |  DFS every mission  LRTA* (learns)  DFS once + stored map |  moves saved vs DFS
           1 |               851           17926                   851 |               0.0%
           2 |              1702           19229                  1318 |              22.6%
           3 |              2553           20602                  1785 |              30.1%
           5 |              4254           23223                  2718 |              36.1%
          10 |              8509           29627                  5053 |              40.6%
          20 |             17018           42836                  9722 |              42.9%
          50 |             42545           79517                 23729 |              44.2%


saved.
```

B.3 Task 2 – follow-up on bushy mazes and LRTA* convergence (t2b_output.txt)

```
FOLLOW-UP A  'bushy' mazes (randomised Prim) - perfect mazes with many short dead ends, far / middle / near targets, 30 mazes per size
     size  cells dead ends            target    opt | DFS moves     CR |  LRTA* t1     CR | t10 CR
   11x11      49       8.8    near (20th pct)    4.7 |      33.7   7.63 |      20.2   4.60 |   1.00
   11x11      49       8.8     far (farthest)   17.0 |      48.2   2.86 |     104.0   6.17 |   1.00
   21x21     199      32.1    near (20th pct)   12.0 |     149.5  12.65 |     145.7  12.31 |   1.00
   21x21     199      32.1     far (farthest)   39.1 |     187.7   4.86 |     879.7  23.17 |   3.47
   31x31     449      71.3    near (20th pct)   19.5 |     424.2  22.20 |     454.6  23.53 |   1.12
   31x31     449      71.3     far (farthest)   60.7 |     442.4   7.32 |    3573.5  58.78 |   2.59
   41x41     799     127.5    near (20th pct)   27.2 |     634.4  23.34 |    1471.4  54.59 |   6.85
   41x41     799     127.5     far (farthest)   82.1 |     780.7   9.54 |    8152.1  99.69 |   3.60


FOLLOW-UP B  LRTA* convergence over many trials (far target, mean of 20 mazes) - trials needed until CR <= 1.10 and moves at trial 100
  DFS-carved 21x21   CR at trials 1,2,5,10,25,50,100: 17.64, 2.04, 2.03, 2.07, 1.80, 1.23, 1.00   first trial with mean CR<=1.10: 56
  Prim 21x21         CR at trials 1,2,5,10,25,50,100: 20.27, 2.21, 1.64, 2.67, 1.00, 1.00, 1.00   first trial with mean CR<=1.10: 20
  DFS-carved 11x11   CR at trials 1,2,5,10,25,50,100: 4.63, 1.72, 1.51, 1.25, 1.00, 1.00, 1.00   first trial with mean CR<=1.10: 17
```

B.4 Task 3 (t3_output.txt)

```
TASK 3 - PREDICTIONS, computed analytically BEFORE any tree was searched.  Author: Saai Vardhan D (192525205)
Uniform game tree, branching factor b, depth d (root = MAX).  Counts: nodes visited (every call), leaves evaluated, cutoffs.
Reasoning:
 BEST  (best move first at every node): only the 'minimal tree' is searched. Node types: PV node (all b children searched; first child PV,
others CUT),
       CUT node (only 1 child searched, then the rest are pruned = 1 cutoff), ALL node (all b children searched, each a CUT node).
       T1(h)=1+T1(h-1)+(b-1)T2(h-1)   T2(h)=1+T3(h-1)   T3(h)=1+b*T2(h-1)   T*(0)=1      (h = remaining depth)
       leaves = b^ceil(d/2) + b^floor(d/2) - 1 (Knuth & Moore).   cutoffs = number of CUT nodes with h>=1.
 WORST (worst move first at every node, all values distinct): each child improves the bound, so no child is ever refuted -> NO cutoffs,
       all b^d leaves, (b^(d+1)-1)/(b-1) nodes: alpha-beta does exactly the work of minimax.
 RANDOM order: between the two. Knuth-Moore / Pearl: about b^(3d/4) leaves on average. I allow a factor 0.5 .. 2.
 EQUAL leaf values with cutoff when alpha >= beta: every CUT node is refuted by its first child (value <= alpha or >= beta immediately),
       so I predict the same count as the BEST case.
 Minimax always: b^d leaves, (b^(d+1)-1)/(b-1) nodes, 0 cutoffs.


 b  d | minimax nodes   leaves | BEST nodes  leaves  cutoffs | WORST nodes   leaves  cut |   RANDOM leaves b^(3d/4)   range x0.5..x2
 4  4 |          341      256 |         62      31       21 |         341      256    0 |                         64         32..128
 4  6 |         5461     4096 |        268     127       99 |        5461     4096    0 |                        512        256..1024
 4  8 |        87381    65536 |       1098     511      417 |       87381    65536    0 |                       4096       2048..8192
 3  8 |         9841     6561 |        393     161      152 |        9841     6561    0 |                        729        364..1458
 5  6 |        19531    15625 |        491     249      180 |       19531    15625    0 |                       1398        699..2795


=============================================================================================================
OBSERVED  (every alpha-beta result was checked to return the same root value as minimax)
=============================================================================================================


 b  d     tree |  AB nodes  AB leaves  cutoffs | predicted leaves pred nodes pred cut | leaves / b^d | verdict
 4  4     best |        62         31       21 |               31         62       21 |       0.1211 | EXACT match
 4  4    worst |       338        253       19 |              256        341        0 |       0.9883 | DIFFERS
 4  4     lexi |       341        256        0 |              256        341        0 |       1.0000 | EXACT match
 4  4   random |       171        113       31 |          32..128          -        - |       0.4421 | within range  (min 62, max 160 over
100 trees)
 4  4    equal |        62         31       21 |               31         62       21 |       0.1211 | EXACT match
 4  4  minimax |       341        256        0 |
 4  6     best |       268        127       99 |              127        268       99 |       0.0310 | EXACT match
 4  6    worst |      4979       3671      696 |             4096       5461        0 |       0.8962 | DIFFERS
 4  6     lexi |      5461       4096        0 |             4096       5461        0 |       1.0000 | EXACT match
 4  6   random |      1453        937      320 |        256..1024          -        - |       0.2287 | within range  (min 476, max 1341
over 100 trees)
 4  6    equal |       268        127       99 |              127        268       99 |       0.0310 | EXACT match
 4  6  minimax |      5461       4096        0 |
 4  8     best |      1098        511      417 |              511       1098      417 |       0.0078 | EXACT match
 4  8    worst |     62418      45412    11616 |            65536      87381        0 |       0.6929 | DIFFERS
 4  8     lexi |     87381      65536        0 |            65536      87381        0 |       1.0000 | EXACT match
 4  8   random |     11256       7141     2741 |       2048..8192          -        - |       0.1090 | within range  (min 3872, max 9854
over 100 trees)
 4  8    equal |      1098        511      417 |              511       1098      417 |       0.0078 | EXACT match
 4  8  minimax |     87381      65536        0 |
 3  8     best |       393        161      152 |              161        393      152 |       0.0245 | EXACT match
 3  8    worst |      8392       5408     1615 |             6561       9841        0 |       0.8243 | DIFFERS
 3  8     lexi |      9841       6561        0 |             6561       9841        0 |       1.0000 | EXACT match
 3  8   random |      2366       1321      613 |        364..1458          -        - |       0.2013 | within range  (min 614, max 1955
over 100 trees)
 3  8    equal |       393        161      152 |              161        393      152 |       0.0245 | EXACT match
 3  8  minimax |      9841       6561        0 |
 5  6     best |       491        249      180 |              249        491      180 |       0.0159 | EXACT match
 5  6    worst |     16791      13197     2272 |            15625      19531        0 |       0.8446 | DIFFERS
 5  6     lexi |     19531      15625        0 |            15625      19531        0 |       1.0000 | EXACT match
 5  6   random |      3702       2569      768 |        699..2795          -        - |       0.1644 | within range  (min 1463, max 3437
over 100 trees)
 5  6    equal |       491        249      180 |              249        491      180 |       0.0159 | EXACT match
 5  6  minimax |     19531      15625        0 |


DEPTH SWEEP b = 4 : leaves evaluated and effective exponent e = log_b(leaves)/d  (minimax e = 1, best-case e -> 0.5)
 d |  minimax |     best     e |  random (mean of 20)     e | rev-oracle     e |     lexi     e |   equal
 1 |        4 |        4  1.00 |                    4  1.00 |          4  1.00 |        4  1.00 |       4
 2 |       16 |        7  0.70 |                   13  0.91 |         16  1.00 |       16  1.00 |       7
 3 |       64 |       19  0.71 |                   42  0.90 |         64  1.00 |       64  1.00 |      19
 4 |      256 |       31  0.62 |                  121  0.86 |        250  1.00 |      256  1.00 |      31
 5 |     1024 |       79  0.63 |                  360  0.85 |        976  0.99 |     1024  1.00 |      79
 6 |     4096 |      127  0.58 |                  865  0.81 |       3565  0.98 |     4096  1.00 |     127
 7 |    16384 |      319  0.59 |                 2553  0.81 |      12862  0.98 |    16384  1.00 |     319
 8 |    65536 |      511  0.56 |                 6884  0.80 |      49587  0.97 |    65536  1.00 |     511


saved.
```

B.5 Task 4 (t4_output.txt)

```
TASK 4 - PREDICTIONS (written BEFORE any experiment was run)
Game: Connect-K with gravity on a W x H board (K=4).  b = number of non-full columns = W on an almost empty board.
Depth-limited minimax, plain (no pruning, no move ordering).  Evaluation function = window-count heuristic + centre bonus (defined in the
report).
N1  Nodes examined for depth d (root = depth 0):   N(d) = 1 + b + b^2 + ... + b^d = (b^(d+1) - 1)/(b - 1)
N2  Terminal evaluations (leaves at the depth limit): L(d) = b^d ; every leaf calls the evaluation function once (or detects a win/draw).
N3  Fraction of nodes that are leaves = b^d / N(d) -> (b-1)/b  (for b = 7: 86 %).  So almost all work is evaluation calls.
N4  On the EMPTY board no column fills and no 4-in-a-row is possible before ply 7, so for d <= 6 the measured counts must equal the
formulas EXACTLY.
N5  Mid-game positions have fewer than b moves (full columns, immediate wins/blocks ending the tree) so b_eff (= nodes^(1/d)) will be
slightly below W.
T1  Time per node in pure Python is 10-60 microseconds (evaluation scans ~69 windows on 7x6).  Acceptability criterion: 1.0 second per
move.
    Prediction: 7x6 board  -> depth 5 fits (about 19,000 nodes), depth 6 (137,000 nodes) does NOT.
                6x5 board  -> depth 5 fits, depth 6 borderline/not.     5x5 board -> depth 6 fits (about 19,500 nodes), depth 7 (97,000)
does not.
Q1  Move quality: depth 1 (greedy on the evaluation function) already beats a random player in > 70 % of 20 games; depth >= 3 wins >= 95 %.
Q2  Each extra ply multiplies time by about b (5-7x); quality gains flatten after depth 3-4 against a random opponent.
```

```
================================================================================================================
OBSERVED
================================================================================================================


X1  EMPTY BOARD: measured vs formula N(d) = (b^(d+1)-1)/(b-1) and L(d) = b^d   (root counted in N)
board                    d |      nodes   formula | leaf evals       b^d | eval calls |   time s  us/node | exact?
5x5 K4                   1 |          6         6 |          5         5 |          5 |    0.000      9.8 | EXACT
5x5 K4                   2 |         31        31 |         25        25 |         25 |    0.000      8.9 | EXACT
5x5 K4                   3 |        156       156 |        125       125 |        125 |    0.001      7.4 | EXACT
5x5 K4                   4 |        781       781 |        625       625 |        625 |    0.006      7.5 | EXACT
5x5 K4                   5 |       3906      3906 |       3125      3125 |       3125 |    0.030      7.7 | EXACT
5x5 K4                   6 |      19526     19531 |      15620     15625 |      15620 |    0.153      7.8 | differs (early wins / full
columns)
5x5 K4                   7 |      97506     97656 |      77980     78125 |      76508 |    0.709      7.3 | differs (early wins / full
columns)
5x5 K4                   8 |     478366    488281 |     382332    390625 |     378272 |    3.597      7.5 | differs (early wins / full
columns)
6x5 K4                   1 |          7         7 |          6         6 |          6 |    0.000      9.3 | EXACT
6x5 K4                   2 |         43        43 |         36        36 |         36 |    0.000      9.5 | EXACT
6x5 K4                   3 |        259       259 |        216       216 |        216 |    0.002      9.6 | EXACT
6x5 K4                   4 |       1555      1555 |       1296      1296 |       1296 |    0.015      9.7 | EXACT
6x5 K4                   5 |       9331      9331 |       7776      7776 |       7776 |    0.094     10.0 | EXACT
6x5 K4                   6 |      55981     55987 |      46650     46656 |      46650 |    0.569     10.2 | differs (early wins / full
columns)
6x5 K4                   7 |     335701    335923 |     279720    279936 |     274650 |    3.377     10.1 | differs (early wins / full
columns)
6x5 K4                   8 |    1980451   2015539 |    1649820   1679616 |    1632000 |   19.081      9.6 | differs (early wins / full
columns)
7x6 K4 (Connect Four)    1 |          8         8 |          7         7 |          7 |    0.000     14.2 | EXACT
7x6 K4 (Connect Four)    2 |         57        57 |         49        49 |         49 |    0.001     13.7 | EXACT
7x6 K4 (Connect Four)    3 |        400       400 |        343       343 |        343 |    0.005     12.8 | EXACT
7x6 K4 (Connect Four)    4 |       2801      2801 |       2401      2401 |       2401 |    0.037     13.1 | EXACT
7x6 K4 (Connect Four)    5 |      19608     19608 |      16807     16807 |      16807 |    0.255     13.0 | EXACT
7x6 K4 (Connect Four)    6 |     137257    137257 |     117649    117649 |     117649 |    2.210     16.1 | EXACT
7x6 K4 (Connect Four)    7 |     960793    960800 |     823536    823543 |     810504 |   14.588     15.2 | differs (early wins / full
columns)


X2  MID-GAME POSITIONS: 10 random positions (6-10 random plies, not finished), mean nodes and effective branching b_eff = N^(1/d)
board                    d | mean nodes mean evals       b^d | b_eff | mean time s
5x5 K4                   1 |          6          5         5 |  5.90 |       0.000
5x5 K4                   2 |         29         23        25 |  5.41 |       0.000
5x5 K4                   3 |        143        110       125 |  5.22 |       0.001
5x5 K4                   4 |        669        520       625 |  5.09 |       0.006
5x5 K4                   5 |       3118       2323      3125 |  5.00 |       0.024
5x5 K4                   6 |      13834      10377     15625 |  4.90 |       0.108
6x5 K4                   1 |          7          6         6 |  7.00 |       0.000
6x5 K4                   2 |         42         35        36 |  6.45 |       0.000
6x5 K4                   3 |        246        195       216 |  6.27 |       0.002
6x5 K4                   4 |       1389       1131      1296 |  6.10 |       0.014
6x5 K4                   5 |       7952       6164      7776 |  6.03 |       0.071
6x5 K4                   6 |      43241      34381     46656 |  5.92 |       0.370
7x6 K4 (Connect Four)    1 |          8          7         7 |  8.00 |       0.000
7x6 K4 (Connect Four)    2 |         56         48        49 |  7.46 |       0.001
7x6 K4 (Connect Four)    3 |        388        320       343 |  7.30 |       0.005
7x6 K4 (Connect Four)    4 |       2625       2215      2401 |  7.16 |       0.035
7x6 K4 (Connect Four)    5 |      18068      14733     16807 |  7.10 |       0.293
7x6 K4 (Connect Four)    6 |     120530      99842    117649 |  7.03 |       2.009


X3  MOVE QUALITY: minimax (maximiser) vs random player, 20 games (10 as first player, 10 as second), random player seeded 0-19
board                  depth | wins draws losses | win rate | mean time per game s
5x5 K4                     1 |   20     0      0 |     100% |                 0.00
5x5 K4                     2 |   19     0      1 |      95% |                 0.00
5x5 K4                     3 |   20     0      0 |     100% |                 0.01
5x5 K4                     4 |   19     1      0 |      95% |                 0.03
5x5 K4                     5 |   19     0      1 |      95% |                 0.15
6x5 K4                     1 |   20     0      0 |     100% |                 0.00
6x5 K4                     2 |   20     0      0 |     100% |                 0.00
6x5 K4                     3 |   20     0      0 |     100% |                 0.02
6x5 K4                     4 |   19     0      1 |      95% |                 0.08
6x5 K4                     5 |   20     0      0 |     100% |                 0.48
7x6 K4 (Connect Four)      1 |   20     0      0 |     100% |                 0.00
7x6 K4 (Connect Four)      2 |   20     0      0 |     100% |                 0.01
7x6 K4 (Connect Four)      3 |   20     0      0 |     100% |                 0.04
7x6 K4 (Connect Four)      4 |   20     0      0 |     100% |                 0.28
7x6 K4 (Connect Four)      5 |   20     0      0 |     100% |                 1.76


saved.
```

B.6 Task 5 (t5_output.txt)

```
TASK 5 - PREDICTIONS (written BEFORE any experiment was run)
Principle given: remove value a from D(Xi) if some neighbour Xj has no value b in D(Xj) compatible with (Xi=a, Xj=b).
My derived procedure (see report): arcs (Xi,Xj) in a queue; REVISE removes unsupported values; when D(Xi) shrinks, re-queue every arc
(Xk,Xi) with k != j,
because the support of Xk's values may have depended on the removed value; empty domain = wipe-out = no solution; MAC = backtracking that
runs the procedure after every assignment.
I1 Instance A (precedence chain + conflicts, one unary restriction): propagation alone should shrink EVERY domain to one value (solved with
0 search nodes), 0 wipe-outs.
I2 Instance B (A fixed to slot 0; B,C in {0,1}; A,B,C mutually in conflict): propagation alone should detect failure with 0 assignments
(exactly 1 wipe-out).
I3 'Pigeonhole' instance: k+1 mutually conflicting exams, k slots, full domains. Propagation sees nothing at the start (every value has
support), so it cannot
   detect failure alone. Both BT and MAC need about k! assignments; I predict MAC visits fewer nodes than BT (factor 2-6) but both grow
factorially.
I4 'Hidden core' instance: 12 loosely connected exams placed FIRST in the variable order and an unsatisfiable core (5 mutually conflicting
exams, 4 slots) placed LAST.
   Plain BT thrashes (hits the 100,000-node cap). MAC with a static order also thrashes because the failure is found only when the core is
reached.
   MAC + 'minimum remaining values / highest degree' variable ordering reaches the core first and finishes in < 1,000 nodes.
I5 Random timetables (3 slots, average degree 4.2, 20 instances per size n = 10..60): BT nodes grow roughly exponentially in n. MAC cuts
nodes by 1-2 orders of magnitude
   at n = 50-60 but spends more constraint checks PER NODE (propagation cost), so the saving in checks is smaller than the saving in nodes.
```

```
I6 Correctness: no value that belongs to some solution is ever removed (to be argued by reasoning in the report; sanity-checked by brute
force on small random instances).


================================================================================================================
OBSERVED
================================================================================================================


INSTANCE A (propagation alone should solve it)
  propagation: consistent=True  arc revisions=59  values removed=20  constraint checks=151  wipe-outs=0
  final domains: {'Maths': [0], 'Physics': [1], 'Chem': [2], 'Biology': [3], 'English': [3], 'History': [0], 'Art': [3], 'Music': [1, 2]}
  all domains singletons (solved without search): False
  verify solution against ALL constraints: True {'Maths': 0, 'Physics': 1, 'Chem': 2, 'Biology': 3, 'English': 3, 'History': 0, 'Art': 3,
'Music': 1}
  BT       nodes=       8 backtracks=       0 checks=        35 wipe-outs/dead-ends=      0  -> solved
  MAC      nodes=       8 backtracks=       0 checks=       186 wipe-outs/dead-ends=      0  -> solved
  MAC+MRV  nodes=       8 backtracks=       0 checks=       186 wipe-outs/dead-ends=      0  -> solved


INSTANCE B (propagation alone should detect failure)
  propagation: consistent=False  revisions=5  removals=3  checks=9  wipe-outs=1  (0 assignments made)
  domains when the wipe-out was found: {'A': [0], 'B': [], 'C': [1], 'D': [0, 1, 2]}
  BT       nodes=       2 backtracks=       2 checks=         5 wipe-outs/dead-ends=      1  -> no solution
  MAC      nodes=       0 backtracks=       0 checks=         9 wipe-outs/dead-ends=      1  -> no solution
  MAC+MRV  nodes=       0 backtracks=       0 checks=         9 wipe-outs/dead-ends=      1  -> no solution


INSTANCE C  PIGEONHOLE: k+1 mutually conflicting exams, k slots (unsatisfiable); propagation at the root removes nothing


  k=4: 5 exams, 4 slots; root propagation consistent=True, values removed at root=0; k!=24
  BT       nodes=      64 backtracks=      64 checks=       556 wipe-outs/dead-ends=     24  -> no solution
  MAC      nodes=      40 backtracks=      40 checks=       944 wipe-outs/dead-ends=     24  -> no solution
  MAC+MRV  nodes=      40 backtracks=      40 checks=       944 wipe-outs/dead-ends=     24  -> no solution


  k=5: 6 exams, 5 slots; root propagation consistent=True, values removed at root=0; k!=120
  BT       nodes=     325 backtracks=     325 checks=      4405 wipe-outs/dead-ends=    120  -> no solution
  MAC      nodes=     205 backtracks=     205 checks=      5605 wipe-outs/dead-ends=    120  -> no solution
  MAC+MRV  nodes=     205 backtracks=     205 checks=      5605 wipe-outs/dead-ends=    120  -> no solution


  k=6: 7 exams, 6 slots; root propagation consistent=True, values removed at root=0; k!=720
  BT       nodes=    1956 backtracks=    1956 checks=     38166 wipe-outs/dead-ends=    720  -> no solution
  MAC      nodes=    1236 backtracks=    1236 checks=     37800 wipe-outs/dead-ends=    720  -> no solution
  MAC+MRV  nodes=    1236 backtracks=    1236 checks=     37800 wipe-outs/dead-ends=    720  -> no solution


  k=7: 8 exams, 7 slots; root propagation consistent=True, values removed at root=0; k!=5040
  BT       nodes=   13699 backtracks=   13699 checks=    363055 wipe-outs/dead-ends=   5040  -> no solution
  MAC      nodes=    8659 backtracks=    8659 checks=    291305 wipe-outs/dead-ends=   5040  -> no solution
  MAC+MRV  nodes=    8659 backtracks=    8659 checks=    291305 wipe-outs/dead-ends=   5040  -> no solution


  k=8: 9 exams, 8 slots; root propagation consistent=True, values removed at root=0; k!=40320
  BT       nodes=  100000 backtracks=   99992 checks=   3449882 wipe-outs/dead-ends=  36786  -> CAP
  MAC      nodes=   69280 backtracks=   69280 checks=   2539360 wipe-outs/dead-ends=  40320  -> no solution
  MAC+MRV  nodes=   69280 backtracks=   69280 checks=   2539360 wipe-outs/dead-ends=  40320  -> no solution


  k=9: 10 exams, 9 slots; root propagation consistent=True, values removed at root=0; k!=362880
  BT       nodes=  100000 backtracks=   99992 checks=   4349818 wipe-outs/dead-ends=  36786  -> CAP
  MAC      nodes=  100000 backtracks=   99993 checks=   3967032 wipe-outs/dead-ends=  58196  -> CAP
  MAC+MRV  nodes=  100000 backtracks=   99993 checks=   3967032 wipe-outs/dead-ends=  58196  -> CAP


INSTANCE D  HIDDEN UNSATISFIABLE CORE: 12 loosely connected exams first, 5-exam clique last, 4 slots, node cap 100,000
  BT       nodes=  100000 backtracks=   99986 checks=    852508 wipe-outs/dead-ends=  36560  -> CAP
  MAC      nodes=  100000 backtracks=   99987 checks=   2033613 wipe-outs/dead-ends=  57634  -> CAP
  MAC+MRV  nodes=  100000 backtracks=   99989 checks=   2039118 wipe-outs/dead-ends=  57825  -> CAP


SCALING: random timetables, 3 slots, average degree 4.2, 20 instances per n (seeds 0-19), node cap 100,000 per run
  n alg       |     nodes backtracks      checks wipe-outs | solved no soln capped
 10 BT        |       110        106         603        37 |      7      13      0
 10 MAC       |        18         15         611         9 |      7      13      0
 10 MAC+MRV   |         9          6         503         4 |      7      13      0
 20 BT        |      5851       5844       32641      2313 |      7      13      0
 20 MAC       |        56         49        2950        30 |      7      13      0
 20 MAC+MRV   |        14          7        1030         4 |      7      13      0
 30 BT        |     49541      49518      281833     14351 |     12       2      6
 30 MAC       |       295        277        7183       142 |     12       8      0
 30 MAC+MRV   |        25          7        1602         4 |     12       8      0
 40 BT        |     86310      86285      513305     25741 |      2       1     17
 40 MAC       |       243        227       21650       135 |      8      12      0
 40 MAC+MRV   |        39         23        2721         7 |      8      12      0
 50 BT        |     95080      95049      556215     28468 |      1       0     19
 50 MAC       |       920        898       98275       512 |      9      11      0
 50 MAC+MRV   |        49         26        4165         9 |      9      11      0
 60 BT        |    100000      99962      602065     27477 |      0       0     20
 60 MAC       |       560        524       43934       252 |     12       8      0
 60 MAC+MRV   |        69         33        4362         9 |     12       8      0


SANITY CHECK of the correctness argument: 300 random small instances (6 exams, 3 slots, mixed conflict / precedence / unary constraints)
  instances=300  values removed in total=1924  removed values that belong to some solution = 0  (must be 0)


saved.
```

B.7 Task 5 – refined instance A2 and ordering fragility (t5_extra_output.txt)

```
EXTRA 1  Instance A2 = instance A plus one more conflict (Music-Physics) so that propagation leaves NO free choice
  consistent=True revisions=64 values removed=21 checks=161 wipe-outs=0
  final domains: {'Maths': [0], 'Physics': [1], 'Chem': [2], 'Biology': [3], 'English': [3], 'History': [0], 'Art': [3], 'Music': [2]}  all
singletons: True
```

```
  verify: True


EXTRA 2  How fragile is the variable-ordering rescue?  hidden_core(seed) for 20 seeds; node cap 100,000; 'quick' = finished with < 1,000
nodes
seed max free degree |        BT       MAC   MAC+MRV
   0               3 |    100000    100000        40
   1               3 |    100000    100000        40
   2               2 |    100000    100000        40
   3               2 |    100000    100000        40
   4               4 |    100000    100000    100000
   5               4 |    100000    100000    100000
   6               4 |    100000    100000     94156
   7               5 |    100000    100000    100000
   8               3 |    100000    100000        40
   9               6 |    100000    100000    100000
  10               3 |    100000    100000        40
  11               3 |    100000    100000        40
  12               3 |    100000    100000        40
  13               2 |    100000    100000        40
  14               4 |    100000    100000    100000
  15               3 |    100000    100000        40
  16               2 |    100000    100000        40
  17               3 |    100000    100000        40
  18               2 |    100000    100000        40
  19               3 |    100000    100000        40
  finished quickly (<1,000 nodes): BT 0/20   MAC 0/20   MAC+MRV 14/20
```

B.8 Task 6 (t6_output.txt)

```
TASK 6 - PREDICTIONS (written BEFORE any experiment was run)
Representation: a tour is a permutation of the n-1 customers (customer 1..n-1); the depot 0 is fixed at both ends, so rotations of the same
cycle are not counted twice.
Exhaustive search generates every permutation by recursive extension, adds the leg length incrementally, and keeps the cheapest complete
tour.
G1  Number of candidate tours enumerated = (n-1)!   n=6: 120   n=7: 720   n=8: 5,040   n=9: 40,320   n=10: 362,880   n=11: 3,628,800.
    Adding ONE customer multiplies the number of tours (and the running time) by n-1 (about 9-10x for n=10-11).
G2  Running time is proportional to the number of search-tree nodes, which is about e*(n-1)! (sum of k! terms), so time(11)/time(10) should
be about 10 (range 8-13).
    I expect roughly 1-3 microseconds per tree node in pure Python: n=10 about 1-3 s, n=11 about 10-30 s.
C1  Correctness: cities on a circle -> the optimal tour is the polygon (angular order), length = perimeter.  Exhaustive search must equal
this and an independent
    brute-force (itertools.permutations) and a dynamic-programming (Held-Karp) check on random instances.
L1  2-opt hill climbing from ONE random start gets stuck in local minima: for n = 8-11 I predict mean excess 2-8 % over the optimum and the
optimum reached in only 20-60 % of starts.
L2  With 10 random restarts per run the mean excess falls below 1 % and the optimum is reached in most of the 20 runs (>= 60 %) for n <=
11.
L3  Time of local search grows only polynomially (about n^2 per pass), so n = 100 is solved in about a second, while the exhaustive method
would need 99! tours.
D1  Fixed response time chosen: 2 seconds (an interactive dispatcher screen).  Predicted largest n for exhaustive search = 10
customers+depot (n=11 takes several seconds). Switch to local search at n >= 11.


================================================================================================================
OBSERVED
================================================================================================================


C  CORRECTNESS CHECKS
  Circle instance (8 cities on a circle, radius 50, depot = city 0): exhaustive cost = 293.941316   polygon perimeter = 293.941316   equal:
True   tour visits cities in angular order: True
  tour found: [0, 5, 4, 1, 2, 3, 7, 6, 0]   angular order from depot: [0, 6, 7, 3, 2, 1, 4, 5, 0]
  exhaustive vs itertools.permutations brute force on 30 random instances (n = 6..8): disagreements = 0
  exhaustive vs Held-Karp dynamic programming on 10 random instances (n = 8..10): disagreements = 0


G  GROWTH OF THE SOLUTION SPACE  (instance seed 1; time = mean of 3 runs for n<=10, 1 run for n=11,12)
  n customers | tours counted        (n-1)! match |   tree nodes |    time s ratio to previous     tours/s
  5         4 |            24            24  True |           65 |     0.000                       785,306
  6         5 |           120           120  True |          326 |     0.000             2.2x    1,800,171
  7         6 |           720           720  True |        1,957 |     0.000             6.5x    1,666,772
  8         7 |         5,040         5,040  True |       13,700 |     0.003             7.8x    1,495,492
  9         8 |        40,320        40,320  True |      109,601 |     0.027             7.9x    1,516,440
 10         9 |       362,880       362,880  True |      986,410 |     0.280            10.5x    1,297,409
 11        10 |     3,628,800     3,628,800  True |    9,864,101 |     2.811            10.0x    1,291,112
 12        11 |    39,916,800    39,916,800  True |  108,505,112 |    35.002            12.5x    1,140,405


  Extrapolation at the measured speed (3.5 million tree nodes / s):
    n=13: (n-1)! = 4.790e+08 tours  ->  about 3.710e+02 s = 0.103 hours = 1.18e-05 years
    n=15: (n-1)! = 8.718e+10 tours  ->  about 6.752e+04 s = 18.8 hours = 0.00214 years
    n=20: (n-1)! = 1.216e+17 tours  ->  about 9.422e+10 s = 2.62e+07 hours = 2.99e+03 years
    n=25: (n-1)! = 6.204e+23 tours  ->  about 4.806e+17 s = 1.33e+14 hours = 1.53e+10 years


L  LOCAL SEARCH QUALITY vs EXHAUSTIVE OPTIMUM  (3 instances per n, seeds 1-3; 20 independent runs per instance; 'restart' variant = best of
10 random-restart climbs)
  n inst   optimum | single-start: mean excess %   max %  hit opt  ms/run | 10 restarts: mean excess %   max %  hit opt  ms/run
  8    1    318.61 |                        0.16    0.45    13/20    0.02 |                        0.00    0.00    20/20    0.11
  8    2    281.04 |                        0.00    0.00    20/20    0.01 |                        0.00    0.00    20/20    0.12
  8    3    271.79 |                        0.00    0.00    20/20    0.01 |                        0.00    0.00    20/20    0.12
 10    1    344.77 |                        2.31    4.44     3/20    0.02 |                        0.01    0.08    18/20    0.18
 10    2    301.11 |                        0.38    2.57    17/20    0.02 |                        0.00    0.00    20/20    0.19
 10    3    304.80 |                        0.00    0.00    20/20    0.02 |                        0.00    0.00    20/20    0.21
 11    1    348.69 |                        2.01    3.91     9/20    0.02 |                       -0.00   -0.00    20/20    0.23
 11    2    310.31 |                        0.37    2.49    17/20    0.02 |                        0.00    0.00    20/20    0.19
 11    3    310.00 |                        0.17    0.69    15/20    0.02 |                        0.00    0.00    20/20    0.21
 12    1    362.67 |                        1.45    3.32     6/20    0.02 |                        0.08    1.58    19/20    0.23


M  LOCAL MINIMA: n = 10 (instance 1), 2000 independent single-start 2-opt climbs
  distinct local-optimum tour lengths reached: 4;  reached the global optimum 344.7696 in 575 of 2000 starts (28.8 %)
    length  344.7696  (+ 0.00 %)  reached   575 times
    length  345.0295  (+ 0.08 %)  reached   422 times
    length  346.4540  (+ 0.49 %)  reached   391 times
    length  360.0899  (+ 4.44 %)  reached   612 times
```

```
B  LARGER INSTANCES (no exhaustive optimum): nearest-neighbour baseline vs 2-opt, 20 runs of 10 restarts each (seed 7 instances)
   n |    NN tour | 2-opt mean      best     worst  gain vs NN % | s/run (10 restarts)  2-opt evals/run
  20 |      395.6 |      355.6     355.0     365.5          10.1 |               0.001            5,891
  30 |      538.9 |      427.0     426.3     433.6          20.8 |               0.002           16,768
  50 |      601.2 |      528.4     516.4     548.3          12.1 |               0.006           52,097
 100 |    1078.2 |      858.8     845.5     875.9          20.3 |               0.029          264,865


saved.
```

B.9 Task 7 (t7_output.txt)

```
TASK 7 - DIAGNOSE / OBJECTIVE / DESIGN / PREDICTIONS  (written BEFORE the redesign was implemented or any Task 7 experiment was run)


DIAGNOSIS (from Task 4 evidence, hypotheses to be confirmed by measurement)
 H1  ~86 % of the nodes of the plain minimax tree on 7 columns are leaves, and every leaf costs one full evaluation that scans all windows
(69 on 5x5, 69 on 7x6 = 69 windows x 4 cells).
     Prediction: evaluate() accounts for >= 70 % of the running time of plain minimax.  => the cheapest speed-up is to evaluate FEWER
leaves, not to micro-optimise.
 H2  Plain minimax explores every move SEQUENCE, but many sequences reach the SAME position (transposition: columns 3,4 then 4,3 ...).
     Prediction: at depth 6 from the empty 7x6 board the number of DISTINCT positions is between 20 % and 50 % of the 7^6 = 117,649
sequences.
 H3  Plain minimax never uses what it already knows: a move that is provably worse than an earlier alternative is still searched to full
depth (no pruning), and move order is left-to-right (column 0 = worst move first).


OBJECTIVE (measurable)
 O1  Root value of the redesigned search = root value of plain minimax at the same depth (zero mismatches; correctness is non-negotiable).
 O2  At fixed depth 6 on 7x6 mid-game positions: >= 10x fewer evaluation calls than plain minimax, with ALL iterative-deepening work
counted.
 O3  Deepest completed search within the 1.0 s per-move budget of Task 4: plain reaches depth 5 on 7x6 (Task 4: d=5 0.26 s, d=6 2.2 s); the
redesign must reach depth >= 7.
 O4  Memory of the transposition table at depth 7 below 100 MB.


DESIGN (four components, each justified by one hypothesis)
 C1 alpha-beta pruning (H3)                    C2 centre-first move ordering (H3, because central moves are strongest on this board, so
cutoffs come early)
 C3 transposition table with EXACT / LOWER / UPPER flags and Zobrist keys (H2)       C4 iterative deepening, best move of the previous
iteration is tried first (H3)
 Soundness rule: a stored value is used as a value only when it was computed for the SAME remaining depth (win scores contain the remaining
depth); older entries are used only to order moves.


PREDICTIONS
 P1  Evaluation calls at depth 6 relative to plain (mean of 10 mid-game positions):
        alpha-beta alone (natural column order)  : 3x - 8x fewer         alpha-beta + centre ordering : 10x - 30x fewer
        + transposition table                    : a further 1.3x - 2x    full design (+ iterative deepening, all iterations counted): 20x
- 60x fewer
        transposition table ALONE (no pruning)   : 1.5x - 3x fewer
 P2  Components are not additive: ordering is worth more than the TT, because ordering changes the pruning from ~b^(d/2) only in the best
case to close to it.
 P3  Deepest depth within 1.0 s on 7x6: plain 5; redesign 7 or 8.     On 5x5 plain 6; redesign 9 or more.
 P4  TT at depth 7 on 7x6: 100,000 - 600,000 entries, 30 - 150 MB in Python.  (So O4 is probably missed or borderline - to be reported
honestly.)
 P5  At depth 1-2 the redesign is NOT faster than plain (hashing + sorting overhead) : ratio of time 1.0x - 3x slower.  The original stays
preferable for depth <= 2.
 P6  Head-to-head, 0.2 s per move, 7x6, 20 games alternating colours: redesign (depth 6) beats plain minimax (depth 4) in >= 14 of 20 games
(draws count half).
 P7  Zero correctness mismatches in all tests.
 P8  Alpha-beta returns exact values only for the best root move; the other root moves return bounds, so the redesign cannot be used when
the exact value of EVERY move is needed (e.g. to rank or explain moves).


=============================================================================================================
OBSERVED
=============================================================================================================


D1  DIAGNOSIS H2 - distinct positions vs move sequences (7x6, empty board, breadth-first with a set of Zobrist keys)
  depth 1: move sequences 7^1 =         7   distinct positions =         7   ratio =  1.00
  depth 2: move sequences 7^2 =        49   distinct positions =        49   ratio =  1.00
  depth 3: move sequences 7^3 =       343   distinct positions =       238   ratio =  0.69
  depth 4: move sequences 7^4 =     2,401   distinct positions =     1,120   ratio =  0.47
  depth 5: move sequences 7^5 =    16,807   distinct positions =     4,263   ratio =  0.25
  depth 6: move sequences 7^6 =   117,649   distinct positions =    16,422   ratio =  0.14
  depth 7: move sequences 7^7 =   823,543   distinct positions =    54,131   ratio =  0.07


D2  DIAGNOSIS H1 - share of running time spent in evaluate() (plain minimax, 7x6, 10 mid-game positions, depth 4)
  one evaluate() call = 19.5 us;  evaluate calls = 20395;  time in evaluate ~ 0.398 s of 0.485 s total = 82 %


E1  CORRECTNESS - root value of every configuration vs plain minimax, and tie-aware check of the chosen move
  3360 (position, depth, configuration) tests on 3 board sizes, positions with 3-22 plies incl. forced wins/blocks: value mismatches = 0,
chosen move not an optimal move = 0


E2  ABLATION - 7x6, 10 mid-game positions (8-14 plies), fixed depth, means; 'x' = plain / this configuration
depth configuration                 |      nodes      evals  evals x  nodes x |    time s   time x | TT entries  TT hits
    4 plain (Task 4)                |       2462       2040     1.0x     1.0x |     0.050    1.0x |          0        0
    4 TT only                       |       1823       1417     1.4x     1.4x |     0.035    1.4x |        262       92
    4 alpha-beta                    |        675        479     4.3x     3.6x |     0.012    4.0x |          0        0
    4 alpha-beta + centre           |        374        233     8.8x     6.6x |     0.006    7.9x |          0        0
    4 alpha-beta + centre + TT      |        350        211     9.7x     7.0x |     0.006    8.8x |        104       17
    4 FULL (+ iterative deepening)  |        334        212     9.6x     7.4x |     0.006    9.0x |         76        9
    5 plain (Task 4)                |      16583      13090     1.0x     1.0x |     0.317    1.0x |          0        0
    5 TT only                       |       8409       6105     2.1x     2.0x |     0.151    2.1x |       1215      556
    5 alpha-beta                    |       3563       2588     5.1x     4.7x |     0.063    5.0x |          0        0
    5 alpha-beta + centre           |       1811       1222    10.7x     9.2x |     0.031   10.2x |          0        0
    5 alpha-beta + centre + TT      |       1438        913    14.3x    11.5x |     0.024   13.4x |        329       60
    5 FULL (+ iterative deepening)  |       1048        676    19.4x    15.8x |     0.018   17.6x |        201       31
    6 plain (Task 4)                |     106812      86325     1.0x     1.0x |     2.178    1.0x |          0        0
    6 TT only                       |      30987      21678     4.0x     3.4x |     0.563    3.9x |       4504     3373
```

```
    6 alpha-beta                      |     11239      7714    11.2x     9.5x |    0.194   11.2x |          0        0
    6 alpha-beta + centre             |      4438      2501    34.5x    24.1x |    0.066   33.0x |          0        0
    6 alpha-beta + centre + TT        |      3054      1588    54.4x    35.0x |    0.043   50.2x |        889      294
    6 FULL (+ iterative deepening)    |      2094      1224    70.5x    51.0x |    0.034   64.0x |        441      102


E3  DEPTH REACHED WITHIN THE 1.0 s PER-MOVE BUDGET (deepest COMPLETED search; plain = deepest fixed depth that finishes in <= 1.0 s)
board    position | plain depth plain evals | redesign depth redesign evals  time s  same value at common depth
5x5 K4          0 |           7       50016 |             25          23849    1.00                         True
5x5 K4          1 |           7       29598 |             30           9865    0.45                         True
5x5 K4          2 |           7       27328 |             30           8260    0.38                         True
5x5 K4          3 |           7       28923 |             30           4024    0.27                         True
5x5 K4          4 |          10       60391 |             30           1973    0.12                         True
6x5 K4          0 |           6       38764 |             14          37220    1.00                         True
6x5 K4          1 |           6       15859 |             30            877    0.09                         True
6x5 K4          2 |           6       28052 |             12          45009    1.00                         True
6x5 K4          3 |           6       27868 |             15          36965    1.00                         True
6x5 K4          4 |           6       27822 |             30           6301    0.47                         True
7x6 K4          0 |           5       15375 |             10          33946    1.00                         True
7x6 K4          1 |           5       15554 |             10          34323    1.00                         True
7x6 K4          2 |           5        6776 |             30           7726    0.80                         True
7x6 K4          3 |           5       15121 |             10          32048    1.01                         True
7x6 K4          4 |           5       14765 |             16          27635    1.00                         True
7x6 K4          5 |           5       10326 |             26          13448    1.00                         True
7x6 K4          6 |           5        8935 |             23          15617    1.00                         True
7x6 K4          7 |           5       14644 |             10          35594    1.00                         True
  5x5 K4: mean plain depth 7.6   mean redesign depth 29.0
  6x5 K4: mean plain depth 6.0   mean redesign depth 20.2
  7x6 K4: mean plain depth 5.0   mean redesign depth 16.9


E4  TRANSPOSITION-TABLE MEMORY (7x6, 5 mid-game positions, FULL design, fixed depth; Python tracemalloc peak)
  depth 6: mean entries       392   mean peak memory     0.1 MB  (max 0.1 MB)   mean time 0.10 s   bytes/entry ~ 138
  depth 7: mean entries       920   mean peak memory     0.1 MB  (max 0.2 MB)   mean time 0.29 s   bytes/entry ~ 125
  depth 8: mean entries     1,763   mean peak memory     0.3 MB  (max 0.4 MB)   mean time 0.44 s   bytes/entry ~ 142


E5  WHEN THE ORIGINAL IS BETTER - shallow depth, 7x6, 10 mid positions, mean of 20 repetitions (time per decision in ms)
  depth 1: plain    0.152 ms   redesign    0.231 ms   redesign/plain =  1.52
  depth 2: plain    1.168 ms   redesign    0.641 ms   redesign/plain =  0.55
  depth 3: plain    7.071 ms   redesign    2.198 ms   redesign/plain =  0.31
  depth 4: plain   49.538 ms   redesign    6.035 ms   redesign/plain =  0.12


E6  HEAD-TO-HEAD, 7x6, 0.2 s per move: redesign (iterative deepening to the budget) vs plain minimax (depth 4 = deepest depth that fits 0.2
s), 20 games, alternating colours
  redesign: 13 wins, 2 draws, 5 losses  (score 14.0 / 20);  mean completed depth of the redesign = 15.9


saved.
```

B.10 Task 7 – table memory for the hardest positions (t7_extra_output.txt)

```
T7-extra: TT size for the hardest positions (those that used the whole 1.0 s budget), 7x6, and for 5 s budgets
  budget 1s position 0: depth  8  nodes   17,664  evals    9,729  TT entries    3,587  peak    0.6 MB  value 207
  budget 1s position 1: depth  8  nodes   16,896  evals    9,917  TT entries    3,192  peak    0.6 MB  value 28
  budget 1s position 3: depth  8  nodes   17,920  evals    9,738  TT entries    3,940  peak    0.5 MB  value -98
  budget 1s position 7: depth  8  nodes   20,224  evals   11,299  TT entries    3,934  peak    0.5 MB  value 97
  budget 5s position 0: depth 10  nodes   96,256  evals   46,664  TT entries   22,385  peak    4.5 MB  value 171
  budget 5s position 1: depth 11  nodes   99,328  evals   48,319  TT entries   21,136  peak    3.1 MB  value 322
  budget 5s position 3: depth 10  nodes   95,232  evals   45,467  TT entries   23,241  peak    4.5 MB  value -108
  budget 5s position 7: depth 10  nodes   96,256  evals   51,334  TT entries   18,309  peak    2.7 MB  value 108
```